The RushTeX Book: Guarded Direct Lowering and the Work a Page Needs

RushTeX Project* The Contributors

2 November 2026

Abstract

RushTeX is a compiler from LaTeX to PDF written in safe Rust. Its goal is output identical to pdfLaTeX's, glyph for glyph and pixel for pixel, made with as little work as that result allows. This document is the project's own book. Each section takes one part of typesetting and answers four questions: what the thing is, what pdfLaTeX does to produce it, what RushTeX does instead, and what is gained. It is also the project's growing test. Every construct it describes is used in its own text, the whole must compile on RushTeX's native path, and the result must match pdfLaTeX exactly. The book follows a document from its source to the PDF: how the source is read and a single pass decided on, how each part of a page is set, how packages become data, how the file is written, how a second compile after an edit is made fast, and how exactness is proved. An appendix lists what each section exercises.

Contents

1 What RushTeX is **6**
1.1 The invariant: identical output 7
1.2 Two paths 7
1.3 How to read this book 7

2 The work behind a page **8**
2.1 Guarded Direct Lowering 9
2.2 The nine rules 10

3 Reading the source **11**
3.1 Characters, category codes and tokens 11
3.2 What RushTeX reads instead 11
3.3 Accents, letters and punctuation 12
3.4 Inverted marks, pounds and other signs 12
3.5 Verbatim text 12

*The native path described here is the work of 22 October to 2 November 2026.

3.6 Upper case . . . 13
3.7 Capitals with the primitive . . . 13
3.8 Text read from other files . . . 13

4 One pass: the pre-scan and references in memory 13
4.1 The pre-scan chooses the path . . . 14
4.2 Headings made by the document's own macros . . . 14
4.3 Cross references in one pass . . . 14
4.4 Bibliographies from BibTeX . . . 15
4.5 Lists of figures and tables . . . 15
4.6 Definitions in the preamble . . . 15
4.7 Index entries . . . 16

5 Fonts 16
5.1 Metrics and the font selection scheme . . . 16
5.2 The font commands . . . 17
5.3 Kerns, ligatures and the word table . . . 17
5.4 Text symbols . . . 18
5.5 The T1 font encoding . . . 18
5.6 Latin Modern . . . 19
5.7 Margin kerning and font expansion . . . 19

6 Paragraphs 19
6.1 Boxes, glue and penalties . . . 19
6.2 The space factor . . . 20
6.3 Quotation marks . . . 20
6.4 Hyphenation, badness and demerits . . . 20
6.5 Breaking lines and adding space by hand . . . 21
6.6 Breaks by hand . . . 21
6.7 Line spacing . . . 21
6.8 Paragraphs set apart . . . 21

7 Boxes, rules and space 22
7.1 Boxes of a given width . . . 22
7.2 Boxes of a fixed height . . . 23
7.3 Boxes, glue and penalties by hand . . . 24
7.4 Struts that start a paragraph . . . 24
7.5 Leaders . . . 24

8 Lists and displayed text 24
8.1 Lists . . . 24
8.2 Lists with keys . . . 25
8.3 Short list labels . . . 26
8.4 Description lists with keys . . . 26
8.5 Description lists measured by enumitem . . . 27
8.6 The enumerate package . . . 27

8.7 Labels of the document's own . . . 27
8.8 Quotations and verse . . . 28
8.9 Alignment of text . . . 28
8.10 Environments of the document's own . . . 29

9 Headings, running heads and counters 29
9.1 The title . . . 29
9.2 Sectioning . . . 29
9.2.1 A third-level heading . . . 29
9.2.2 Chapters in the report class . . . 30
9.3 Running heads . . . 31
9.4 Running marks . . . 31
9.5 Marks of the document's own . . . 31
9.6 Counter forms . . . 31
9.7 Counters and lengths of the document's own . . . 32
9.8 Footnotes and citations . . . 32

10 Pages, floats and columns 33
10.1 The page builder and the output routine . . . 33
10.2 Floats . . . 33
10.3 Ending a page . . . 34
10.4 Floats set here . . . 36
10.5 Float styles and new float types . . . 36
10.6 Algorithms . . . 37
10.7 Captions . . . 38
10.8 Columns . . . 38
10.9 Column breaks . . . 39
10.10 Columns in a box . . . 40

11 Tables 40
11.1 Alignments and the tabular preamble . . . 40
11.2 Rules, spacing and spanning cells . . . 40
11.3 Paragraph columns . . . 41
11.4 Tables of a given width . . . 41
11.5 Declarations in the preamble . . . 42
11.6 Column types of one's own . . . 42
11.7 Columns centred and set from the bottom . . . 43
11.8 Table rows kept in a command . . . 43
11.9 Verbatim text in a table . . . 43
11.10 Colour in table entries . . . 43
11.11 Notes under a table . . . 44
11.12 Long tables on one page . . . 45

12 Pictures **45**
12.1 The picture environment . . . 46
12.2 Drawings . . . 47
12.3 TikZ libraries and pictures in a colour . . . 47
12.4 Pictures in the text . . . 48
12.5 Nodes . . . 48
12.6 Points, paths and loops . . . 50
12.7 QR codes . . . 53
12.8 Coloured boxes . . . 53
12.9 Trees in TikZ . . . 54

13 Mathematics **55**
13.1 How a formula is set . . . 55
13.2 Inline formulae . . . 55
13.3 Displays and equations . . . 55
13.4 Displayed derivations . . . 56
13.5 Arrays of equations . . . 56
13.6 Symbols built from others, and bold symbols . . . 57
13.7 Math fonts after a formula in a box . . . 57
13.8 Symbol and alphabet packages . . . 58
13.9 Fractions with a slash . . . 58
13.10 Cancelled terms . . . 59

14 The amsmath package **60**
14.1 What amsmath changes . . . 60
14.2 Text, operators, dots and delimiters . . . 60
14.3 Equations and their tags . . . 60
14.4 Aligned equations in one pass . . . 61
14.5 Gathered equations . . . 62
14.6 Equations too long for a line . . . 62
14.7 Aligned blocks inside a formula . . . 62
14.8 One equation over several lines . . . 63
14.9 Cases, matrices and arrays . . . 64
14.10 mathtools and calc . . . 65
14.11 Display breaks . . . 66
14.12 Tags made of formulas and commands . . . 66

15 Theorems **67**
15.1 Theorems within theorems, titles with commands . . . 68
15.2 The end mark inside a display . . . 68

16 Packages and their options **69**
16.1 How a package becomes native . . . 69
16.2 Package options . . . 69
16.3 Packages loaded and not used . . . 69
16.4 Page layout . . . 70

16.5 Colour . . . 70
16.6 Mixing printing colours . . . 70
16.7 Web addresses . . . 70
16.8 Quantities and units . . . 71
16.9 Degree and Celsius signs without siunitx . . . 71
16.10 Numbers wider than their column format . . . 72
16.11 Units written out and long numbers . . . 72
16.12 Program listings . . . 72
16.13 Numbered lines, short code and other languages . . . 73
16.14 Comments and strings in listings . . . 73
16.15 Named listing styles . . . 73
16.16 Listings read from a file . . . 74
16.17 Preprocessor lines and tabs in listings . . . 74

17 Writing the PDF **75**
17.1 Shipping a page . . . 75
17.2 Fonts in the file . . . 75
17.3 Compression . . . 75

18 Speed: each piece of work once **76**
18.1 Compiling again after an edit . . . 76
18.2 Several cores . . . 77

19 Exactness and the testing framework **77**

20 Measurements **77**
20.1 The benchmark documents . . . 78
20.2 Fixed cost and cost per page . . . 78

21 Current limits **79**

A What this document tests **79**

B Feature probes **81**
B.1 More of amsmath and mathtools . . . 81
B.2 Scaled, resized and rotated boxes . . . 82
B.3 Cells of several lines . . . 83
B.4 Spaces after commands and float barriers . . . 83
B.5 Commands with a default argument . . . 83
B.6 The title read again . . . 83
B.7 Switches of the document's own . . . 84
B.8 Internal macros of the document's own . . . 84
B.9 Headings of the document's own . . . 84
B.9.1 A heading with *own* macros . . . 84
B.10 Anchors, links and running heads . . . 85
A heading past the depth set in the body . . . 85
B.10.1 A heading numbered again . . . 85

B.11 Tags read ahead, tags in multline, and old font commands 85
B.12 Bare alignment commands, one-token definitions, math in bookmarks 86
B.13 Conditions with ifthen 86
B.14 Page styles of the document's own 86

C References **86**

List of Figures

1 The stages of Guarded Direct Lowering, from the characters of the source file to the bytes of the output, in the order in which the data flows through them in a single pass. 34
2 A short caption. 34
3 A starred figure. 35
4 A figure placed with `H`. 36
5 A caption made with `captionof` in a minipage. 38
6 Pictures: a PDF page scaled to a width, and a PNG image scaled by a factor. 45

List of Tables

T1 Benchmark documents of the native path 33
T2 Feature status at the time of writing. 34
T3 A table placed with `H`. 36
T4 Its neighbour, a table. 38
T1 The preamble letters of `tabular` used in this document. 40
T1 Instructions of one run, measured on 25 October 2026. 78
T2 Fixed and per-page cost, measured on 25 October 2026. 79
T1 A table numbered within its section 85

List of Algorithms

1 Euclid's algorithm 37

List of Programs

10.1 Euclid's algorithm for the greatest common divisor. 36

1 What RushTeX is

A typesetter turns a stream of characters into pages. The author writes paragraphs of prose, and the typesetter decides where lines break, how much space separates words,

where a page ends and where each figure goes. TEX, written by Donald Knuth, makes these decisions so well that its rules became the standard of scientific publishing, and LATEX is the macro language most authors use on top of it. pdfLaTeX is the program that runs LATEX on the pdfTeX engine and writes PDF directly. RushTeX keeps the rules and their results exactly, and changes how the work is organised.

1.1 The invariant: identical output

The invariant is identical output. The same glyphs stand at the same positions, measured in scaled points, the unit of $1/65536$ of a point in which TEX does all of its arithmetic. Every rule that decides a position is taken from the published sources: `tex.web` for the engine, and `latex.ltx` and the package sources for LATEX. The code cites the section it implements, so that each step can be checked against its origin. Nothing is approximated, and nothing is corrected after the fact: a result that differs by one scaled point is a failure, because it would move everything that follows.

1.2 Two paths

RushTeX has two ways to compile a document. The *native path*, the subject of this book, runs LATEX's commands as Rust functions over an exact port of the TEX core. The *engine path* is a complete TEX engine, also written in Rust, that runs the real LATEX format and packages from an installed TEX Live. It takes every document the native path cannot prove it handles, and it produces the same pages. A document that brings its own package or class file always runs on the engine path, because its macros are the author's and are not ported.

The two paths differ in what they need and in how fast they are. A native document needs nothing but the program and its own files: the fonts, the metrics and every number the classes and packages set are carried in the program, and no LATEX macro is interpreted. An engine document needs TEX Live and runs LATEX's macros as pdfLaTeX does, only faster. Which path a document takes is decided once, before typesetting starts, by the pre-scan of Section 4, and the choice is for the whole document.

Today the native path takes the standard classes (article, report, book and letter), amsart, the KOMA-Script classes and memoir, most of the LATEX kernel, and the packages templates load most: among them amsmath, amsthm, graphicx, xcolor, hyperref, geometry, fancyhdr, enumitem, booktabs, tabularx, longtable, multicol, natbib, siunitx, listings, tcolorbox and a growing part of TikZ.[1] The rest run on the engine path, with the same output.

1.3 How to read this book

The sections follow a document from the source file to the PDF. Section 2 gives the whole journey at once: the sixteen kinds of work pdfLaTeX does, and the architecture, Guarded Direct Lowering, that decides what RushTeX does at each of them. Section 3

[1] Of the 758 documents in RushTeX's census of real-world templates (theses, reports, letters, CVs, posters and papers), 303 compiled natively when this was written. The census ranks the commands that block the rest, and the work follows that ranking, the commands that free the most documents first.

reads the source, and Section 4 explains the pre-scan, which chooses the path and lets a single pass do what pdfLaTeX needs several runs for.

Sections 5 to 15 take the typesetting model one part at a time, in the order in which a page is built from its pieces: fonts, paragraphs, boxes, lists, headings and counters, pages with their floats and columns, tables, pictures, mathematics, amsmath and theorems. Each says what the thing is, what pdfLaTeX does for it, what RushTeX does instead and what is gained, and then uses it.

The last part is about the program as a whole. Section 16 explains how a package becomes native data and when a document is declined, Section 17 how the PDF is written, Section 18 how a second compile after an edit is made fast and where threads help, Section 19 how exactness is proved, Section 20 what was measured and Section 21 what is not native yet. Appendix A maps every construct this document exercises to the section that uses it.

Commands are written in typewriter type without their backslash, as in `section` or `includegraphics`, and the text around each one uses it, so that reading a paragraph also shows what it describes.

2 The work behind a page

Between the bytes of the source and the bytes of the PDF, pdfLaTeX does about sixteen kinds of work, most of them once or more for every character. The list below follows a character through them, and the description after each step says what RushTeX's native path does there. Four answers recur: a step is *removed* when nothing on the page depends on it; it is done *once* when its result is the same every time; it is done at *build time* when it depends only on fixed data; and it is *kept* when it decides a position, because those rules are the output.

1. Read the byte from the file into the input buffer. Kept: RushTeX reads the source once, from memory.

2. Give the byte a category code and make a token. Once per word: a run of letters in one font is read straight from the source as one word; other characters still become tokens one at a time.

3. Look a command's name up in the hash table. Build time: every name the native path knows is in a generated perfect hash, and is known by a number after its first lookup.

4. Expand LaTeX's macros into primitives. Removed for native commands, which are Rust functions. Some still build short strings of primitives that are read back as tokens, about nine in a hundred of all tokens; removing those is ongoing work.

5. Load the format, the class, the size file and the font definitions. Build time: the class's lengths, the kernel's values, the font definitions and the font metrics are generated into tables carried in the program.

6. Run the font's ligature and kern program on each character. Once per distinct word: the result is stored in a word table, and a repeated word, most of the words of a long text, is copied from it.

7. Append one node per character to the paragraph. Once per word: a word is a single node holding its summed width, height and depth, and it is opened into characters only where something must look inside.

8. Add up widths in the first line-breaking pass. One width per word instead of one per character; the algorithm is kept.

9. Hyphenate every word of a paragraph whose first pass fails. Once per word: a word's hyphenation points are remembered, and only the words whose hyphenation could depend on their neighbours are opened.

10. Run the second, hyphenating pass, and an emergency pass if needed. Kept exactly, since it decides the lines.

11. Pack each line into a box, measuring its contents again. Per word, from the stored sums.

12. Build pages, place insertions and run the output routine, whose float handling is written as macros over lists of box names. The page builder is kept; the output routine and the float queue are Rust code over a small array of box numbers.

13. Write the auxiliary file, and run the whole document again until labels, citations and the contents settle. Removed: references are resolved in memory in the same pass.

14. Walk each page to write its text, selecting a font for each character. Per word, with the font made current once per word.

15. Read, decrypt, subset and encrypt the font programs. Build time for reading: the programs, decrypted, and their glyph names are carried in the program. Each document's subset is cut as pdfTeX cuts it.

16. Compress each page's content stream. Kept, with a fast compressor using fixed codes.

So item 1 and the core steps are kept, and the rest are removed, done once, or moved to build time, each only where the result cannot change. The sections that follow explain each step in detail, and Section 19 explains how the choices are checked.

2.1 Guarded Direct Lowering

The architecture that organises these choices is Guarded Direct Lowering. Its name states its method: a guard decides once whether the native path can take a document, and each command is then lowered directly to the typesetting lists, with no layer of macros in between. The pipeline has five stages.

- The pre-scan reads the whole source once and proves that every command it uses is one the native path implements unchanged.
- The direct compiler turns each command into calls of typed node builders.
- The node builders append boxes, glue, kerns and penalties to the current list, as the corresponding primitives of TEX would.
- The exact core breaks paragraphs, builds pages and lays out mathematics:
  - its arithmetic is TEX's own, integer for integer,
  - and its results are never adjusted afterwards.
- The shipping stage turns each finished page into PDF operators.

Figure 1, in Section 10, draws the same chain. A document the guard cannot prove, for example one that loads its own package file, goes whole to the engine path. The guard never sends half a document: a document is native from its first byte to its last, or not at all.

2.2 The nine rules

Nine rules keep the native path honest. Each was adopted because breaking it either cost work for no visible result or risked a page that differs.

No TEX text A native command builds nodes, not strings of primitives to be read again; the author's own text and typed closing steps are the exceptions.

Numbers, not names A name known in advance is a number at run time.

Build time first Whatever depends only on fixed data is computed when the program is built.

One guard A document is accepted or refused once, before it runs.

Fixed results The core's results are final; nothing corrects them later.

General rules A behaviour comes from the rule that defines it, never from a case written for one document.

Data and mechanisms Tables hold the data, code holds the mechanisms.

Least movement Data is moved as little as possible.

Only the unobservable A step is dropped only if nothing can observe that it is missing; a document that could observe it goes to the engine.

The first two rules now hold everywhere on the native path. No native command writes TEX source to be read again: theorem heads, equation numbers, the contents, the accents and the operators of mathematics are built by node builders, and the only tokens a native command still places in the input are the author's own text and the braces, math shifts and alignment marks around it. The names a native command needs are looked up once, when the job starts, and the commands themselves are numbered when the program is compiled.

The last rule is what makes the shortcuts safe. A word is kept as one node because a native document cannot look inside a box with TEX's box-reading primitives: the guard refuses those, and a document using them runs on the engine.

3 Reading the source

3.1 Characters, category codes and tokens

TEX does not read words; it reads characters, and gives each one a *category code*, a number from 0 to 15 that says what the character means. A letter has category 11, a space 10, a brace 1 or 2, the backslash 0. Characters of category 0 start a *control sequence*, a command name such as `section`. The reader turns characters into *tokens*: a character token is its code with its category, and a control sequence token points into a hash table of all names. pdfLaTeX does this for every character of the document, and looks every command name up in that table each time it is met.

Ten characters are special in LATEX. Seven of them are typed as commands when they are meant literally: the ampersand &, the per cent sign %, the dollar $, the hash #, the underscore _ and the braces { and }. A text font such as Times has no brace glyphs, so LATEX borrows them from the math symbol font: its font definition file for Times in the symbol encoding sends every shape there, and a family with no such file, such as the typewriter type of {`this`}, falls back to the encoding's default family by LATEX's rule for undefined shapes. RushTeX's font table holds the same definitions and its selection follows the same rule, so the braces come from the same font. The tilde makes a tie, a space where no line may break, so a tied name such as Ludwig van Beethoven never splits across lines.

3.2 What RushTeX reads instead

RushTeX's native path reads the source once, from memory. A run of letters in one font is taken straight from the bytes as one word, without a token per character; only the other characters become tokens one at a time. Command names are looked up in a perfect hash generated when the program is built: a table with no collisions, so that a name is found in one probe, after which the command is known by its number. A token is four bytes, and a command is a Rust function, not a macro whose text must be read again.

The gain is that most characters of a text never become tokens at all. A paragraph of prose such as this one is read as a sequence of words and spaces, and only its commands and punctuation go through the token reader.

3.3 Accents, letters and punctuation

Accents are commands that place a mark over or under the next letter: café, Gödel and naïve carry acute and diaeresis marks, and the full set is à, á, ô, ñ, ā, ż, ő, ă, č and å. A dotless ı and ȷ take an accent without their own dot. Letters outside the basic alphabet are commands too: ß, æ, œ, ø, Æ, Œ and Ø, and a cedilla, a ring over a capital and a barred l are built from boxes: garçon, Ångström, å, łódź and Ł. pdfLaTeX expands each accent command into a box built by the TEX primitive `accent`, which centres the mark using the slant of the font; RushTeX calls the same primitive's rule directly.

The same letters may be typed directly in UTF-8: café, Gödel, naïve, Erdős, Škoda, señor, Straße, Français, Ångström and Łódź, with "curly quotes", an en dash in 1–2, an em dash—like this—and an apostrophe in it's. Each character stands for the kernel's command for it, so it is set exactly as the command would be. Text symbols have commands of their own as well: 20°, 3×4, ±1, ©, ® and Name™.

Vertical space comes in three standard amounts, `smallskip`, `medskip` and `bigskip`, which add three, six or twelve points of stretchable space between paragraphs; this paragraph follows a medium one. The old declaration *em emphasises* and switches back *inside itself*, the formula $a^2 + b^2$ is set in math wherever it is typed, and a rule _____ can be measured against the column.

The plain TEX breaks `smallbreak`, `medbreak` and `bigbreak` end the paragraph, invite a page break there with a penalty, and add the same amount unless a space at least as large is already there, as at the end of this paragraph, where the big break replaces the small skip.

The paragraph after it.

Quotations are typed as "double quotes", the apostrophe in don't is a plain character, and the dashes – en and — em – come from ligatures of the font, as described in Section 5.3. Sentences that trail away end with … set as a unit.

3.4 Inverted marks, pounds and other signs

A character typed in UTF-8 is two to four bytes. pdfLaTeX makes the first byte active and expands a macro that reads the rest and looks up the command the character stands for. RushTeX looks the bytes up once, in a table of the kernel's 349 characters generated when the program is built, and runs the command directly; the glyph is the same. Spanish opens a question with ¿ and an exclamation with ¡, ¿así? ¡Sí! (¿ and ¡ by name). A price is £5 or £10; a rate 5‰; the ordinals 1ª and 2º; the superscripts ¹, ² and ³; and the signs ¦ and ¤. In bold: **¿Qué? £5‰**.

3.5 Verbatim text

Verbatim text is set exactly as typed, with its spaces and special characters kept. Its category codes are changed so that no character is special, which is why it cannot be read ahead like ordinary text. This is the command that runs the testing framework's gate, each line in its own `\verb` (this manual loads `tcolorbox`, whose `verbatim` package redefines the environment):

```
parity-testing zero --engine --pixels --all
# every glyph and pixel; $ & % # _ { } stay as typed
```

Short inline code is written with `\verb`, as in `\section{Title}` or `x = a, b; y--z`. The text between the two delimiters is read as it stands in the file, in the typewriter font, without ligatures and with French spacing: `e.g. this`. The starred form shows each space as a visible space: a␣b␣c.

3.6 Upper case

Headings, running heads and title pages often set their text in capitals. pdfLaTeX's `MakeUppercase` expands its text fully and then changes the case of what is left; RushTeX changes the case as the kernel's code does, letter for letter. `\MakeUppercase` changes the case of its text inside a group, as THE KERNEL'S TEXT CASE CODE DOES, accented letters of the input included: **PRÁTICAS, AÇÃO, ÜBER, ÉLAN**. A command, math or a letter whose capital is two letters (ß) inside it still goes to the engine.

3.7 Capitals with the primitive

The primitive `\uppercase` turns the letters of its text into capitals as it reads them, before any command in the text runs, and leaves the commands themselves alone: **VEGETARIAN • WEEKNIGHT • 30 MINUTES**. Its braces only delimit the text, so a change of font inside it still lasts to the end of its own group: A *SLANTED* WORD AND **BOLD** ONES.

3.8 Text read from other files

A long document is often split into files, one per chapter, read with `\input` or `\include`. pdfLaTeX opens a file read with `input` when the command is met, in the middle of typesetting. RushTeX puts the file's text in its place before the pre-scan, so the pre-scan checks it as part of the document and the prediction of Section 4 counts its headings and labels. The paragraph goes on in the file: This paragraph is kept in a file of its own, `figures/inputpart.tex`, and read where the main file says `\input{figures/inputpart}`, as a thesis keeps each chapter in its own file. Its last line ends where the file ends. The text after it goes on in the same paragraph.

4 One pass: the pre-scan and references in memory

pdfLaTeX discovers a document while it typesets it. When it sets the first page it does not know what a later page will say about it, so it writes what it learns to auxiliary files and asks to be run again. RushTeX reads the whole source once before typesetting. That reading, the *pre-scan*, has two jobs: it proves that the document can take the native path, and it predicts every number the document will print, so that a single pass sets the pages as pdfLaTeX's last run sets them.

4.1 The pre-scan chooses the path

Every native command is one row of a table compiled into the program: its name, its arguments, the function that sets it, the modes in which it may be used and the package that provides it. The pre-scan, the prediction of numbers and the reading of arguments are all derived from that row, so a command made native needs no guard of its own. The pre-scan walks the source once and checks each command against the table, each package and option against the list of native ones, and each assumption the native commands make, such as a category code left alone or an internal macro not redefined.

A document that fails any check runs on the engine path from its first byte: one that loads a package or class file of its own, uses a command not yet ported, or redefines a kernel internal after `makeatletter`. The reason is printed when the variable `RT_NATIVE_WHY` is set, and this document's gates require it to print that the native job was accepted. A check that can only be made during the run, such as a predicted number that turns out wrong, also sends the whole document to the engine. The gain is that a native document runs no LaTeX macro and needs no TeX installation, while every other document still compiles, with the same pages.

4.2 Headings made by the document's own macros

The heading of this section comes from a macro of this document, `\rtpart`, whose text is `\section{#1}\label{#2}`. The contents are read from the source before the body runs, so a heading inside a macro would hide from them; a macro whose text is plain (its arguments, no optional one, no internal names) is expanded there as TeX would expand it, and the contents list this section as any other. pdfLaTeX learns the heading only when the macro runs and puts it in the contents on the next run; RushTeX knows it before the first page.

4.3 Cross references in one pass

A label records a number at the place it stands, and a reference prints it anywhere. pdfLaTeX writes every label to an auxiliary file during the run and reads that file at the start of the next run, so a reference to a later label, a forward reference, prints two question marks until the document is compiled again. The table of contents works the same way through a second file. A document with a contents list usually needs three runs: the first writes the entries, the second sets the contents and moves every page after it, and the third sees the moved page numbers. This document needed three runs of pdfLaTeX when it was last measured.

RushTeX writes no auxiliary file and runs once. The pre-scan reads every label's value off the source before typesetting starts, by following the counters the native commands step, and each label checks the predicted value when it is reached: a wrong prediction sends the document to the engine, so it can never print a wrong number. The contents list is set from the headings the pre-scan read; its page numbers stand in boxes of a fixed width, and the real digits are put in when the pages are known, without laying

out any page again. So Section 21, a forward reference, prints its number here in the first and only run.

A reference to a page uses `pageref`: the list of the kinds of work in Section 2 began on page 8. A page reference to a place not yet written is not native yet, since its width could move the lines before it.

A command the preamble defines is read through when its text numbers nothing. One whose text holds only unnumbered headings, such as a question heading made of a title and its points, numbers nothing either, except under the hyperref package: each such heading takes the next anchor of its own series, so the prediction steps that series once for every heading in the command's text.

The cleveref package prints the kind of thing a label names along with its number: a reference written `\cref{eq:euler}` reads eq. (14.1), and `\Cref` starts it with a capital, as in Theorem 15.1. Several labels of one kind are sorted and runs of three or more are joined into a range, so sections 1 to 3 and 5 names four sections, and `\labelcref` prints the numbers alone: 7 and 8. cleveref finds the kind in a second auxiliary entry that each label writes, holding the counter the last `refstepcounter` stepped; the kernel's repair for cleveref rewrites that entry at every label from `@currentcounter`, so a lemma numbered by the theorem counter is a theorem to it: theorem 15.2. RushTeX keeps the counter beside the label's value as each native command steps it, and takes a later label's kind from the counter the pre-scan predicted for it, so the names are right in the first run, with or without hyperref.

4.4 Bibliographies from BibTeX

The kernel's `bibliography` writes the database names to the auxiliary file and inputs `\jobname.bbl`; `bibliographystyle` only writes to the auxiliary file. The native path reads no auxiliary file: before the pre-scan it puts the text of the `.bbl` file where `bibliography` stands and drops `bibliographystyle`. The references at the end of this book come that way, from `master.bbl` [2]. pdfLaTeX needs a run to write the auxiliary file, a run of BibTeX, and two more runs before every citation is resolved; RushTeX reads the `.bbl` file BibTeX left and resolves each citation in the pass that sets it.

4.5 Lists of figures and tables

`listoffigures` and `listoftables` are set in one run, as the contents are: their entries are the captions read from the source, each with its short title, and their page numbers are filled in when the pages are known. The `float` package's `\listof{type}{title}` lists a kind of float it declared the same way, and `\listofalgorithms` is the `algorithm` package's `\listof{algorithm}`; the list of algorithms and the list of programs after the contents are made so, from the floats of Sections 10.5 and 10.6.

4.6 Definitions in the preamble

An author's own short commands are part of most papers. This document's preamble defines one, `gdl`, which prints Guarded Direct Lowering, with `newcommand`; it

redefines the roman family with `renewcommand` and declares an operator name with `DeclareMathOperator`. The plain TEX form works as well: `def` gives `half`, so that $\frac{x}{2}$ halves its argument, and a `def` in the body, like this one, is read the same way. A definition whose text uses only native commands is native too: RushTeX expands it as TEX would. A document that redefines a macro the native commands read themselves, such as `thesection` or `labelitemi`, is set by the engine, which reads every definition. Numbers held in macros are read as TEX reads them: this document's preamble sets `baselinestretch` to 1 and `textfraction` to 0.2, their usual values.

Such a command also matters to the pre-scan. To set forward references and the contents in one pass, the pre-scan reads the headings and labels of the body ahead of typesetting, following every counter the native commands step. An author's command could step one it cannot see, so the pre-scan reads each definition of the preamble first: a command whose text holds only commands that step no counter, as `gdl` does, is passed by; any other stops the reading there, and a document that needs the rest of it goes to the engine path. Every predicted number is still checked when its label is reached, so a wrong prediction can only send the document to the engine, never print a wrong number.

A command of the document's own whose text is a centred line numbers nothing, so the contents are still read before the body runs:

* * *

So does pandoc's `\tightlist`, given with `\providecommand` and two length assignments, at the head of a list:

- one item,
- and another, with no space between.

4.7 Index entries

An index entry is written to the index file and set nowhere in the text: the words around it keep their single space, and a command of the document's own may carry one, as Guarded Direct Lowering does here. The index itself is made by a separate program between runs, so a single run has none to print.

5 Fonts

5.1 Metrics and the font selection scheme

A font, to TEX, is a metric file: for every character a width, a height, a depth and an italic correction, plus a program of ligatures and kerns and a few parameters such as the width of a space. The glyph shapes are needed only when the PDF is written. LATEX chooses fonts with its font selection scheme, which describes a font by five attributes: encoding, family, series, shape and size. A command such as `textbf` changes one attribute; then `selectfont` combines the five into a name, loads the family's font definition file when it is first used, finds the shape, and loads the metric file.

pdfLaTeX does all of this at run time, in macros, every time the font changes: the attributes are strings, the name is built by expansion, and a cache of loaded fonts is searched by that name. RushTeX generates the tables at build time instead: 354 metric files and 630 font shapes are read when the program is built, checked against LaTeX's own choice, and carried in the program as arrays. A font change is then an index into those arrays.

Font packages that change the whole document are tested on their own documents, since loading one here would change every page. mathpazo is one: Palatino for the text and the Pazo fonts for mathematics, with its own math codes for the upright symbols, a narrower medium space between binary operators, and script sizes of its own (7.6 pt scripts in 10 pt text). RushTeX loads it as a generated table, like the kernel's and amsmath's, and its `sc` option switches to Palatino with real small caps.

5.2 The font commands

This document is set in Times, chosen with one line of the preamble that redefines the roman family, while its sans serif labels and its `file names` keep the Computer Modern families. Emphasis is shown with *italic* words, strong statements with **bold**, names with SMALL CAPITALS, and an *explicit italic* phrase where a term needs one. The other shape and series commands are *slanted*, upright, medium and roman type.

Declarations act to the end of their group, so **this phrase is bold by declaration**, *this one italic*, *this one slanted*, THIS ONE IN SMALL CAPITALS and `this one in typewriter`, each ending at its brace; sans serif, medium, upright and roman work the same way. The older two-letter forms of LaTeX 2.09, **bold**, *italic*, *slanted*, SMALL CAPITALS, sans serif, `typewriter` and roman, reset the other attributes as they did then.

A group limits a declaration: inside these braces the text is bold, and a word such as *this* still becomes italic within it. Outside the group the text returns to the roman of the paragraph.

Sizes are declarations too, from tiny, script size, footnote size and small through normal size to large, Large, LARGE, huge and Huge. A size sets the font and the baseline distance together, which is why this line is taller than its neighbours.

Raised and lowered text, such as the 1st of a date or the H_2O of a formula written in text, is set at the script size of the surrounding type in the current shape: **bold**$^{\mathbf{b}}$ stays bold.

A size need not be one of the class's. `fontsize` with its baseline distance, then `selectfont`, sets any size the font's shape holds: this phrase is thirteen points on sixteen, and this one eight and a half on ten. RushTeX takes such a size when the font definition file lists it or a range holding it, as Times' does for every size; a size that LaTeX would replace by the nearest one it has, with a warning, still goes to the engine.

5.3 Kerns, ligatures and the word table

A kern is a fixed space between two boxes that never stretches or shrinks. Fonts carry kerns for pairs of letters whose shapes would otherwise look badly spaced: in AVAWAY

the font pulls each A and V or W together, and the typesetter inserts the kern from the font's table each time the pair meets. A font's italic correction, added after a slanted letter, is also a kern.

A ligature is a single glyph that replaces a sequence of letters. In first and flight the letters f and i, and f and l, become one glyph each, so the dot and the ascender do not collide with the hook of the f. Which pairs form ligatures depends on the font: Times has fi and fl, Computer Modern also ff, ffi and ffl. The dashes of Section 3 are ligatures too, of two and three hyphens.

pdfLaTeX runs the font's ligature and kern program on every character, as it arrives. RushTeX runs it once for each distinct word in each font, and keeps the result in a *word table*: the glyphs, the kerns and the summed width, height and depth. A word met again, which is most of the words of a long text, is copied from the table. The program's result depends only on the letters and the font, so the copy is exact.

5.4 Text symbols

Symbols that the text fonts lack, such as the degree sign or the euro, live in the text companion encoding, TS1. pdfLaTeX loads a TS1 font definition file when a symbol is first used; RushTeX's font tables already hold the TS1 shapes, so a symbol is one index.

`lipsum`, loaded in templates for dummy text, sets nothing until its command is used. The `comment` package's environment drops its lines whole, as if they were not in the source, so this sentence goes on without a gap. The `verbatim` package, loaded before it here, sets its `verbatim` environment as the kernel does, line for line. `textcomp` and `dcolumn` load as tables and change nothing until used: °, ™, § and † come from the TS1 fonts, also inside a table cell: a† / b‡ .

More text symbols of the TS1 encoding: 5 µm at 20 °C, 3 ■, € 10, ¥ 7, ¼, ¾, ¬, a–b, № 4 and *f*. The spacing accents and arrows that UTF-8 text brings, written as characters or by name, are TS1 symbols the kernel checks against the font family's sub-encoding (`\tc@check@symbol`): ´, ¨, ˘, ˇ, `, ¯, and the arrows ↑, ↓, ←, →. pdfLaTeX expands that check for each use; RushTeX keeps each symbol as a slot and a sub-encoding level, and picks the family's font or the Computer Modern substitute directly.

The sign of `\copyright` is the text companion font's: Copyright © 2026.

5.5 The T1 font encoding

`\usepackage[T1]{fontenc}` is native. It sets the encoding of every text font to T1 (the EC fonts drawn by cm-super), where an accented letter is one character: é, à, ô, ü, ñ, ç, å and š become precomposed glyphs that hyphenate as letters (caractéristiques, télécommunications), while ő, ğ and ǵ keep the accent over the letter. The symbols \, {, }, $, _, | and § come from the text font itself. This document is set in OT1; the gates compile it a second time with T1 loaded, and both must match the engine box for box and pdfLaTeX pixel for pixel.

5.6 Latin Modern

The lmodern package, loaded by a quarter of the thesis and report templates, is one such table: it sets `rmdefault`, `sfdefault` and `ttdefault` to the Latin Modern families, and declares the math fonts and alphabets anew. Its math declarations do not add families; `SetSymbolFont` gives the four families every document has, operators, letters, symbols and large symbols, Latin Modern shapes in place of Computer Modern ones, so RushTeX replaces those families' shapes where they are and moves no family number. The font shapes come from the pinned `.fd` files, read once by LaTeX at build time; the pin now holds `ot1lmtt.fd` too, the typewriter family in the OT1 encoding. A text family whose `.fd` file is not in the tables declines the document, because LaTeX would read the file and RushTeX cannot; before, it was silently replaced by the default family. The probe `scripts/native_probes/lmodern.tex` sets every shape and a page of mathematics in Latin Modern with T1 and amsmath.

5.7 Margin kerning and font expansion

The microtype package, loaded by a fifth of the essay and newsletter templates, turns on two features of pdfTeX: characters such as hyphens, full stops and the round sides of letters hang a little into the margin, and the letters of a line may be widened or narrowed by up to two per cent so that the spaces between words vary less. Both are properties of a font, set by the package when LaTeX first selects it: a code for each character's protrusion at either margin, and the limits of the font's expansion. RushTeX does not read the package's configuration files at run time. A generator sets each font the native job can select, one selection to a run of the engine, and keeps what the package gave it, so that a native selection installs the same codes from a table; a font the table lacks sends the document to the engine. The package also places explicit kerns of its own: before the first character of a list item, of a footnote and of a contents entry, and around the parentheses of an equation number, each the margin kern that character would have; in verbatim text both features are switched off. This document does not load microtype, since it would move every line of it; the probes `scripts/native_probes/microtype.tex` and `scripts/native_probes/microtype-patches.tex` are set with it.

6 Paragraphs

6.1 Boxes, glue and penalties

Everything on a page is a box. A character is a box with a width, a height and a depth, taken from the metric file of its font. A line is a horizontal box holding the boxes of its characters, and a page is a vertical box holding the boxes of its lines. Between the boxes lies glue: a space with a natural width that may stretch or shrink, so that a line of words can be made exactly as wide as the measure.

A penalty is a number that says how bad it would be to break at a point. A large positive penalty forbids a break, a large negative one forces it, and the values in between let the author and the class steer the algorithms. The tie of Section 3 is a space with a penalty of ten thousand.

In pdfLaTeX each character of a paragraph is a node of its own, linked to the next. In RushTeX a word is one node, holding the summed sizes from the word table, and it is opened into characters only where something must look inside: at a hyphenation point chosen by the line breaker, or when a line is written to the PDF.

6.2 The space factor

Word spacing in a justified line is not constant. The stretch a line receives is shared among its interword glue, and the space after a full stop is wider than the space between words, because each character carries a space factor that the following glue reads. A full stop after a capital letter, as in J. R. Smith, does not end a sentence, so the author writes a backslash space where a sentence does continue after one, as in Mr. Smith, and an at-sign command where a capital does end one, as in NASA. Small fixed spaces are typed with `thinspace` or a backslash comma, as in 10 km and 5 kg.

6.3 Quotation marks

The csquotes package types quotation marks from their meaning rather than from the characters an author would type. The command `enquote` wraps its argument in the outer marks of the style, as in “a quoted phrase”, and a quotation inside a quotation takes the inner marks, as in “she said ‘yes’ at once”. The starred form begins with the inner marks: ‘single’. Inside a quotation the punctuation keeps the space factors the package gives it, so the space after a sentence that ends in a quotation stays the wider one: “Done.” So it is. Between a mark and the word next to it the package puts a small marker of glue, which breaks the kern the font would have made there, and then puts the kern back as glue, measured from the font’s own table, as in “AV” and “Vo”. RushTeX types the same marks, markers and kerns. Without babel the style is American English; with babel it is the style of the main language, British English with single outer marks.

6.4 Hyphenation, badness and demerits

Long words that do not fit at the end of a line are hyphenated at points found by Liang’s patterns [4], a compact table built from a word list that says where a word may be divided: typesetting, hyphenation, algorithm and reproducibility break at reasonable places. Each candidate line has a badness, from zero to ten thousand, that measures how far its glue is stretched or shrunk, and a line’s demerits combine its badness with the penalty of its break. The paragraph builder of Knuth and Plass [1] chooses the breaks with the least total demerits over the whole paragraph, first without hyphenation and, when that fails, again with it.

pdfLaTeX hyphenates every word of a paragraph when the first pass fails, and does it again for the same word in the next paragraph. RushTeX remembers each word’s hyphenation points the first time, and its first pass adds one width per word instead of one per character. The algorithm itself, its passes and every one of its tests are kept exactly, because they decide the lines.

6.5 Breaking lines and adding space by hand

Explicit line breaks suit addresses and verse:
the first line ends at the double backslash,
and the second line starts afresh.
This paragraph was started with `noindent`, so it has no indentation, and the gap of in this sentence was inserted by hand. The next paragraph follows after of extra room.

A box made with `mbox`, such as this phrase, is never broken across lines.

The paragraphs that follow are indented as usual: every paragraph starts with an indent box, and `noindent` only removes it.

6.6 Breaks by hand

The author may also steer the line and page breakers directly. Each of these commands leaves a penalty or a discretionary hyphen in the list, a number the breakers weigh like any other, and RushTeX leaves the same ones. A line may be ended here
and a break asked for with a strength from 0 to 4, or kept from breaking at a space. A word hyphenated by hand, handmade, breaks only there, and the preamble's list of exceptions gives the breaks of the made-up word triphonogrammatic. A page break may be kept away too, in small letters if wished. This is note 2 of the subsection. This page may run one line longer than the others, and a space half an indentation wide reads `\parindent` itself.

6.7 Line spacing

Theses are often set with more space between lines. The setspace package changes `baselinestretch` and selects the font again, so that every baseline distance is multiplied; RushTeX reads the stretch as TEX reads the macro and sets the same distances.

Inside a `spacing` environment of setspace the stretch is one and a half: the baselines of this paragraph are further apart than those around it, and the environment ends with a correction for the skip it adds.

The `onehalfspace` environment sets the stretch the class size asks for, here 1.25 for ten points, with a paragraph long enough to wrap.

The text after both environments is set at the normal spacing again.

The kernel's `\linespread` sets the same stretch without the package, taking effect at the next `\selectfont`; in the preamble that is the start of the document. This paragraph is spread by 1.4 inside a group, and runs over two lines.

6.8 Paragraphs set apart

Letters and CVs often load parskip, which marks a new paragraph with space instead of an indent: `parskip` becomes half the normal line distance, with two points of stretch, and `parindent` zero. The package then makes the space between paragraphs the only

space in lists: in the outer three levels the space above a list and between its items is zero and the space between paragraphs within an item is `parskip` itself; `small` and `footnotesize` keep their own outer lists, as their `@listi` is not the one the package redefines. Every heading's space above, and a displayed heading's space below, is less one `parskip`, since the paragraph that follows adds it back. RushTeX sets these values when the package loads and checks them at each list and heading. Its patch of the table of contents (and the lists of figures and tables) is ported too: inside the list's group the space last added is replaced by itself plus `parskip`, and `parskip` is zero for the entries. The patch of amsthm's theorem heads is not, so parskip with amsthm sends the document to the engine (`scripts/native_probes/parskip.tex`).

7 Boxes, rules and space

A box is the unit of everything TEX places: a character, a line, a page. Most of the time the line and page breakers make the boxes, but an author can make one by hand, of a given width or height, framed or coloured, raised or lowered, and fill the space around it with glue or leaders. LATEX's box commands are macros over TEX's `hbox` and `vbox`; RushTeX's are Rust functions that build the same boxes with the same dimensions, so no macro is expanded.

7.1 Boxes of a given width

A paragraph can also be set inside a box of its own width. This `parbox` is three be-
tenths of the text width and aligned by its first line,
second one centred on the
side a line, as LATEX centres a The `minipage` environment does the same for longer box by default.
text, with paragraphs, lists and tables inside:

The left column of two minipages set side by side. Each is a box as tall as its text, so the taller one sets the height of the line.

The right column. Inside a minipage the text width is the box's width, and the first paragraph is not indented.

- A list inside starts afresh at the first level.

RushTeX builds both boxes as LATEX does, from `\@iiiparbox`: a vertical box at the given width, its paragraph shape restored, then a `vtop`, `vbox` or centred box around it. A box with a fixed height follows in Section 7.2; a footnote inside a minipage still goes to the engine.

A frame is a box too: this phrase sits in `fbox`, three points of space inside a rule of four tenths of a point, and a framed parbox holds a whole paragraph the same way. With xcolor loaded, as here, the frame is drawn after the box by xcolor's own `boxframe`, and

RushTeX draws it the same way. xcolor's `colorbox` puts a block of colour behind its box, as in this highlighted phrase, and `fcolorbox` adds a frame in a second colour: a framed white box. The block is a rule as large as the box plus its separation, laid under the text. Colours can also be given as web colours and mixed: this box is twelve parts of the colour 174A6E to eighty-eight of white, and these words are half the blue of this document and half black. xcolor works such mixtures out with its own fixed-point arithmetic, dividing to five decimals and moving decimal points as text, and RushTeX repeats each step, so the numbers written into the page are the same to the last digit.

Label: a value set after a box of fixed width, then centred , right, over p and a plain box; a box of no width lets its text overlap what follows.

7.2 Boxes of a fixed height

A `minipage` or `parbox` may be given a height after its position and then an inner position, which places the text inside that height: `[t][2cm][c]` is a box two centimetres tall, aligned on the line by its top, with its text centred in it.

A framed box with a height and no inner position keeps its text at the top.

NOTICE

text centred in the box

This text sits at the foot of its box.

A timetre tall parbox one centre and a stretched sit on the line beside an empty height, as tall as its, and a box within a box: outer, inner text.

The text of a `parbox`, a footnote or a margin note is the argument of a long macro, so a blank line there is a paragraph break, as in this box, whose first paragraph ends here. A colour Its second starts after the blank line. may be defined in the body outside any group, like this rose, and lasts from there on.

A declaration may be used as an environment: these two lines are centred by `\begin{centering}`, whose end is only the end of its group.

A size or a font declaration works the same way, as this footnote-size sentence with **a bold phrase** in it does. In text, an underlined phrase is a formula holding its box, as the kernel's `\underline` makes it outside math.

The kernel's `\fill` is a skip of zero plus one fill: left middle right.

A `raisebox` lifts its text, up or down by half its height, and may give it another height and depth, as this one that takes no room above the line. A `framebox` is a framed

makebox: centred, at the left, half again as wide.

7.3 Boxes, glue and penalties by hand

Authors reach for TEX's own commands: a box, spread out, raised and lowered text, a lap and a llap, an en space, a kern, a rule, and penalties. The kernel's rlap, llap, null, nobreak, allowbreak, break, sloppy and fussy are made of these primitives alone, and run as their definitions.

A box set in vertical mode, after a vskip.

A paragraph set under sloppy, with its tolerance of 9999 and an emergency stretch of 3em, so that a line with a long word such as pdfTeX-compatibility is spaced out rather than left overfull.

7.4 Struts that start a paragraph

A \strut between paragraphs starts one, as \unhcopy\strutbox does in vertical mode: the paragraph indent comes first, then the strut's rule. memoir builds every head and foot slot so, \parbox[b]{w}{\raggedright\strut ...\strut}: left slot right

7.5 Leaders

Leaders are glue that is drawn: a rule, or a box repeated, as wide as the glue becomes when the line is set. The kernel's hrulefill and dotfill are leaders with infinite stretch, and RushTeX builds the same glue, so the dots of every row stand at the same places.

Leaders fill a line's rest with a rule or with dots, as forms and tables of contents do:
Signature _____
The first chapter 3
Date _____ and place .....

8 Lists and displayed text

8.1 Lists

LATEX builds every list, and every displayed block of text, on one environment of the kernel, list, with a handful of lengths: the left margin, the space between items, the space above and below the list, and the width of the label. Each list environment sets those lengths from the class's tables for its depth, then sets each item's label in a box hung into the left margin. pdfLaTeX does this with macros that are expanded anew for every list and every item. RushTeX's list environments are Rust functions that read the same lengths from tables generated from the class, and build the same boxes and glue.

- An itemised list marks each item with a bullet.

- Lists nest, and each depth has its own label and margin:
  - the second level uses a dash,
    - and the third an asterisk.
- The spacing around a nested list is smaller than around the outer one.

1. An enumerated list numbers its items.
2. Its numbers can be referred to, and nested lists number by letter:
   (a) this item is number 2(a),
   (b) and this one 2(b).

Item 2a above shows how a nested label combines the numbers of its levels. A description list sets a word in bold as each item's label, as the nine rules of Section 2.1 are set.

8.2 Lists with keys

Templates rarely keep the kernel's lists as they are. The enumitem package lets each list, or every list of a kind, take keys for its spacing, margins and labels. pdfLaTeX parses the keys with macros each time a list starts; RushTeX reads them once and sets the same lengths before the list is built.

This document loads the `enumitem` package. With no keys on its lists it makes the same lists as the kernel, save one `nobreak` before each label; its `description` differs more: each label is set with `labelsep` zero and followed by `hfil`, and the item indent becomes `labelsep` less the left margin, so the label starts 5pt further right. The native path builds both, as in the description lists of this document.

Its keys on `enumerate` and `itemize`, in the optional argument or kept by `\setlist` in the preamble, are native too: the vertical lengths (`topsep`, `itemsep`, `parsep`, `partopsep`) and `nosep`, `noitemsep`, `nolistsep`; the horizontal ones with a value (`leftmargin`, `labelsep`, `labelwidth`, `itemindent`, `rightmargin`, `listparindent`); `leftmargin=*`, for which enumitem measures the label with the counter at its widest (`0`, `m`, `viii`) and makes the margin the label's width and `labelsep`; and `label`, whose `\arabic*` and its kin stand for the list's counter and whose whole text becomes what a `\ref` to the item prints. A value is read as enumitem's own assignment reads it, so it may be an expression such as `\dimexpr\linewidth-2\fboxsep`, which takes the assignment's closing `\relax` as its end; the native job checks that nothing of the value is left after it. RushTeX reads the keys in enumitem's order: `\setlist` for every list, for every list at the depth, for the environment and for the environment at its depth, then the list's own. This document's preamble has `\setlist[enumerate,2]{itemsep=1pt}`, so every second-level numbered list in it has its items 1pt apart, and the lists below give their own keys:

(i) a first item in lower-case roman numerals, the margin as wide as the label `(viii)` and `labelsep`,

(ii) a second, holding a list of its own:

Case A: the words around `\Alph*` are copied, the letter counts,

Case B: and this list's `itemsep` comes from the preamble, its right margin from an expression.

The label text, without the outer list's, is what a reference prints, and RushTeX reads it from the source in the same single pass as every other reference: items (i) and Case A:. A list with `nosep` has no space around or between its items:

- one,
- two.

8.3 Short list labels

With enumitem's `shortlabels` option a list's first key may be its label, the counter written as `1`, `a`, `A`, `i` or `I`:

a) the first, labelled a),

b) the second, with more keys after it:

   (i) inside
   (ii) and again.

– an itemize label given the same way.

A list may start at another number and set its labels flush left:

4. the fourth item, its label at the left of its box,

5. and the fifth.

8.4 Description lists with keys

enumitem's `description` takes the horizontal keys with a value or `!` (the length enumitem computes) and the styles. The standard style boxes its label:

Morgan, Alex a label narrower than its 30 mm box.

A label much wider than its box runs on into the item's text.

The style `sameline` puts the text at the left margin and the label unboxed:

Calibration the label fits in front of the text.

Protocol and review a wider label pushes the text to the right.

The style `nextline` starts the text on a new line when the label is wider:

Viscosity fits.

An extremely wide label
does not fit, so the text starts below it.

A second paragraph of the same item.

8.5 Description lists measured by enumitem

A `*` value asks enumitem for the length from the widest label. A `description` has no label of its own (its `\@itemlabel` is empty), so with `leftmargin=*` the label width is less than nothing, minus `\labelsep`, and the left margin comes out as the negative item indent, here zero:

Route the destination of a request.

Rework a request returned to the queue.

With `labelwidth=*` the label width is what is left of the margin:

Rollout a staged release with criteria for going back.

The `\setlist` keys for every list and for `description` come first, as enumitem's `\enit@preset` sets them.

8.6 The enumerate package

The `enumerate` package leaves a list without an optional argument as the kernel makes it. With one, such as `[(i)]`, each `A`, `a`, `i`, `I` or `1` outside braces stands for the counter in that style, the rest is copied, and the left margin of that depth becomes the width of the label at seven plus `labelsep`. It is native too, but it replaces the same `enumerate` that enumitem does, so this document loads enumitem, the one far more templates use, and the enumerate package is tested in a document of its own.

8.7 Labels of the document's own

Many documents choose their own list marks with `\renewcommand{\labelitemi}{--}`, and pandoc writes `\def\labelenumi{\arabic{enumi}.}` at the top of every numbered list. In LaTeX an item's label is whatever `\labelitem`*n* or `\labelenum`*n* expands to when `\item` sets it in its box. RushTeX's lists make the class's labels themselves, without a macro; when the document has given one of those names a macro, the list puts that macro in the label's box instead and the macro expands there, at the same moment as in LaTeX, so it may read `\theenumi` or `\roman{enumii}`, which RushTeX answers from the list's own counters. The gain is that these documents stay on the fast path with their own marks:

- an item marked with a dash,
    - and one inside it, marked with a circle;

a) a numbered item labelled with a letter,

    1.1 inside it a label built from both counters,

b) and item 1, whose reference keeps the kernel's `\theenumi`.

A document may also redefine the counter's text, `\theenumi`, which changes both the label (the kernel's `\labelenumi` is `\theenumi.`) and what a reference prints. RushTeX then reads the macro's text, as it reads the document's own counters, to make the reference:

A. an item numbered with a capital letter,

and its reference, item A, prints the same letter.

8.8 Quotations and verse

Three environments set text in from both margins. A short quotation uses `quote`, a longer one of several paragraphs `quotation`, which indents each paragraph, and poetry `verse`, whose lines are broken by hand.

> Every glyph is compared by its font, its character and its position; every rule and path by its coordinates; every link, outline entry and colour by its value.
>
> A document passes only at zero shift: no glyph moved by any amount, no pixel different.
>
> A difference of a single scaled point in one line's glue counts as a failure, because it would move every glyph that follows.

A box, a glue, a penalty,
a line that breaks where it should be;
the page is built, the float is set,
and not a point is shifted yet.

8.9 Alignment of text

Text may be centred, or set flush against either margin, as an environment or as a declaration that ends with its group.

Zero shift, every page,
for every document of the corpus.

The corpus holds 85 documents from CTAN, each screened with pdfLaTeX first.

The gate passes when all 85 are identical.

The declaration form, `centering`, centres the lines of the paragraph it ends, and leaves the surrounding text alone.

The declaration `raggedright` stops the lines from being stretched to the full measure, so that the right margin is ragged. It is what the `flushleft` environment uses inside.

And `raggedleft` sets the lines against the right margin, with a ragged left one.

8.10 Environments of the document's own

`newenvironment` names a begin text and an end text. Both are expanded where the environment starts and ends, so RushTeX treats them as it treats a command of the document's own (Section 4): native if every command in them is.

Note. An environment the preamble defines with `newenvironment`: its begin and end texts run around this paragraph.

Text after the environment goes on.

9 Headings, running heads and counters

A document's structure is numbered: headings, equations, figures, theorems and the document's own counters. This chapter shows how headings are set, how the running heads read back what the headings leave, and how counters are stepped and printed. The references that print these numbers before they are reached are the subject of Section 4.

9.1 The title

The title of this book carries a `thanks`, marked with the footnote symbol $*$, and two authors separated by `and`.

9.2 Sectioning

The sectioning commands, from `section` through `subsection` and `subsubsection` to `paragraph`, number the heading, set it in its font, leave the space above and below it, write an entry for the table of contents and forbid a page break just after it. In pdfLaTeX each is one macro of the kernel, `@startsection`, called with six arguments that the class supplies. RushTeX's headings are one Rust function that takes the same six values from the class tables.

9.2.1 A third-level heading

A third-level heading is still numbered in the article class, and still listed in the contents.

A paragraph heading. The fourth level is a run-in heading: its title starts the paragraph, in bold, and is not numbered or listed. The fifth level, `subparagraph`, is the same heading moved in by the paragraph indent (`\hskip\parindent` before its title). Its contents line is past the depth the contents show, and with hyperref both levels carry the anchor of an unnumbered heading in front of the title.

A subparagraph heading. This one sits in by the indent.

A heading with no text of its own.

A second one at once. The first heading's paragraph holds its title alone: LaTeX starts the paragraph and ends it before the next heading, and a list after a run-in heading starts that paragraph the same way.

A list follows.

- its first item comes after the heading's line.

9.2.2 Chapters in the report class

The report class is native too. Its state at the first token of the document, every parameter, font and list setting, is a table written by `rushtex-gen class --class report` and checked word by word against pdfLaTeX's, exactly as article's is; the two tables differ only in where the figure and table counters live. What the class code defines otherwise is taken from a comparison of `report.cls` with `article.cls`, one macro at a time.

A chapter is one more heading level above the section. `chapter` starts a new page, gives that page the plain style, keeps floats off its top, and sets the head as `@makechapterhead` does: fifty points down, "Chapter" and the number in `huge` bold, then the title in `Huge` bold, with no page break before the first paragraph and no indent at its start. Because the chapter is level 0 and the section level 1, the rules RushTeX already had give the class's numbering unchanged: the first three levels are numbered (report's `secnumdepth` is two), a section's number is the chapter's number, a point and its own, and stepping a chapter resets every level below it. The equation, figure, table and footnote counters start again in each chapter, and from the first chapter on an equation, figure or table number carries the chapter's number first, as in *Figure 2.1*; one-pass references read those numbers as the pre-scan predicts them.

The table of contents begins with a starred chapter, on a page of its own, and its three levels are set as article sets the same three: a chapter's entry in bold, as a section's is in article, a section's and a subsection's dotted, with the indents of article's subsection and subsubsection, since report shows two levels below the chapter where article shows three below the section. Only the chapter entry differs in its spacing: it is preceded by a plain vertical skip of one em, not one merged with the skip before it, and by a penalty of -301, and followed by a penalty of 301 that keeps it with its first section (`scripts/native_probes/report-toc.tex`).

What report changes and RushTeX does not take yet sends the document to the engine whole: the title page form of `maketitle`, `part`, `appendix`, the lists of figures and tables, the bibliography, `numberwithin`, theorems numbered within a counter, and the hyperref and fancyhdr packages, whose anchors and marks are still article's. The probe `scripts/native_probes/report-chapters.tex` runs a report with chapters, starred chapters, sections, floats, equations, footnotes and references through both paths and compares every page.

The abstract of a report (or of an article with the `titlepage` option) is a page of its own: the class calls `\titlepage` inside the environment, puts a vertical fill above and below, and centres the bold `\abstractname` with a small penalty before it and an

unbreakable one after; the text follows in ordinary paragraphs, and the page counter is set to one at both ends, as for a title page. RushTeX builds the same tokens.

A thesis usually opens with a `titlepage` environment, in either class. It is the classes' definition, ported as it is: a new page with the empty page style, the page counter set to one at the start, and at the end another new page and, in a one-sided document, the page counter set to one again, so the page after the title is page 1. The probe `scripts/native_probes/titlepage.tex` is a thesis title page of that kind.

9.3 Running heads

A running head is set by the output routine after the page is broken, from texts the preamble gives and marks the page holds.

This page carries a running head from the fancyhdr package, given with `thispagestyle{fancy}`; the other pages keep the plain style. fancyhdr sets its head as three paragraphs, one flush left, one centred, one flush right, each as wide as the text and boxed by `parbox`, with a rule underneath, and its foot the same way above the page number. RushTeX writes those boxes from the six texts the preamble gives, so the head is the one pdfTEX makes, down to the strut at the end of each part. fancyhdr also redefines `sectionmark` and `subsectionmark`, so every numbered heading in this document now leaves two marks in the page for a head to read back, even though no head here reads them; the marks are made the same way. Under `pagestyle{fancy}` the contents and the references leave marks of their own, `CONTENTS` in the contents heading and `REFERENCES` after the references heading, and RushTeX makes those too.

9.4 Running marks

The foot of the page set in the fancy style carries, at the right, `\leftmark`: the last left mark of the page, which fancyhdr's `\sectionmark` sets to the section's number and title in capitals, or the one before the page when the page starts none. `\rightmark` is the first right mark of the page, the subsection's. Both are read back from the page's marks when its head and foot are built, after the page is broken.

9.5 Marks of the document's own

A `\markright` or `\markboth` of plain text puts its marks where it stands, for a head that reads them; under the plain style nothing shows.

9.6 Counter forms

A counter is an integer, and its `the` command decides how it is printed. A document may give `thefigure`, `thetable` or `theequation` a text of its own, in the preamble or in the body: a fixed prefix before `arabic`, as supplementary material does with `S`, the section's number before the counter (`\thesection-\arabic{figure}`, common in theses), or the counter in another form. This book numbers its tables T1, T2 and so on: Table T3 above is one. When a caption or an equation steps its counter, LATEX expands the text into the label, and RushTeX reads the same text with the counters as they stand,

so the caption, the list of figures and every reference carry it. Here the equations are numbered after the section in roman numerals for a while,

$$e^{i\pi} + 1 = 0 \tag{9-i}$$

which equation (9-i) shows; then the text is the one `\numberwithin` gave again. A text with a condition, such as `\ifnum`, still sends the document to the engine, and so does a redefinition inside a group when the contents or a later reference depend on it.

9.7 Counters and lengths of the document's own

A document may make its own counters and lengths. Step 1 has parts a and B; step ii starts them again at 0, and MCMXCIX is the same counter in capitals. The first step was step 1. A counter made within the subsection starts again in each: this is note 1. A counter whose `\the` is redefined as `\thesection.\arabic{rtclaim}` labels claim 9.2 with it: claim 9.1, then claim 9.2 of section 9, numbered IX in capitals. A space as wide as a word: [ ].
Left middle right, the space after the middle twice as stretchable.

9.8 Footnotes and citations

A footnote[2] is numbered automatically, and its text goes to the foot of the same page if it fits. Citations work like references: `cite` prints the label of an entry of the bibliography, such as [2] for the reference manual of TEX, and the entries are listed in Section C. `nocite` lists an entry without printing anything where it stands: pdfLaTeX writes the key to the `.aux` file for BibTeX and keeps the space before it from doubling (`\@bsphack` and `\@esphack`). RushTeX writes no auxiliary file, so it leaves only that space rule, which is the whole of what the page shows. This document loads the `cite` package, which sorts the labels of one citation and shortens a run of three or more with a dash, [1–4] for four entries, separates a note with a smaller penalty, [2, ch. 8], and puts the space before a citation back as it found it, with a penalty that lets a line break there.

Psychology and education theses often load `apacite` for the citations of the American Psychological Association. It redefines `\cite` (with `\citeA` for *Author (Year)* and `\citeNP` with no parentheses), and before BibTeX has run, or when the bibliography file is missing, every key is undefined: the package prints a question mark for the authors and one for the year, `(?, ?)`, with its own punctuation macros (`\BBOP`, `\BBAY`, `\BBCP`, ...) around them and the opening parenthesis in a box of its own. RushTeX builds the same tokens for each command, so a document whose bibliography has not been made yet runs natively; this document keeps the kernel's `\cite` for its own bibliography, so the commands are not shown here.

A document may choose its own footnote marks by redefining `\thefootnote`, which LATEX expands when the note's counter is stepped: `\fnsymbol{footnote}` gives the symbols of the title's notes, `\alph{footnote}` letters, and `\let\thefootnote\relax`

[2] A footnote is an *insertion*: material that the page builder holds apart from the text and places at the foot of the page.

a note with no mark at all, a common way to add a remark or a funding line. RushTeX makes the mark itself when the kernel's `\thefootnote` stands; when the document has given it a meaning, the mark's box holds that macro instead, expanded with the same counter, so the symbol‡ and the unmarked note are set as LaTeX sets them.

10 Pages, floats and columns

10.1 The page builder and the output routine

TeX's page builder moves lines from the paragraphs to the current page one by one, and after each it computes the cost of breaking the page there, from the badness of the page's vertical glue and the penalty at the break. When the page is full, it breaks at the cheapest point found and hands the page to the *output routine*, a piece of code the format supplies. LaTeX's output routine adds the running head and foot, puts the footnotes at the bottom, and decides which floats go on this page. RushTeX keeps TeX's page builder exactly, since it decides where pages end, and implements LaTeX's output routine in Rust with the same tests in the same order.

10.2 Floats

A float is a table or figure that the page builder may move away from the place where it appears in the source, so that it never splits across a page break or leaves a large gap at the foot of a page. The placement specifier says where it may go: `h` for here, `t` for the top of a page, `b` for the bottom and `p` for a page of floats, tried in that order.

LaTeX's output routine places floats with a queue of eighteen slots and a set of fractions: at most seven tenths of a page may hold top floats, three tenths bottom floats, and at least one fifth must remain text. In LaTeX the queue is a macro holding a list of box names, rebuilt whenever a float moves; RushTeX keeps it as an array of box numbers and makes the routine's tests in the same order, so every float lands where pdfLaTeX puts it.

Table T1 appears before the text that discusses it, so the reader meets the reference first and the float later.

Benchmark	Pages	Identical
Mathematics	20	yes
Tables	29	yes
Prose	61	yes

Table T1: Benchmark documents of the native path, measured in Section 20.

The documents of Table T1 compile natively and match pdfLaTeX. The next float is a figure with a caption long enough to wrap onto a second line, placed at the top of a page by its specifier.

‡A note marked with `fnsymbol`.
A note with no mark.

Source → Pre-scan → Builders → Core → PDF

Figure 1: The stages of Guarded Direct Lowering, from the characters of the source file to the bytes of the output, in the order in which the data flows through them in a single pass.

Figure 1 shows the stages described in Section 2.1. Its caption is set as a paragraph because it runs over two lines; a short caption is centred. A table placed at the bottom of a page is specified with `[b]`.

A float that may go anywhere is written `[htbp]`, as the next figure is.

`figure` Text placed in a box

Figure 2: A short caption.

Figure 2 is referred to after the float itself. The starred forms, `figure*` and `table*`, span both columns of a two-column page; on a page of one column, as here, they behave as top floats.

10.3 Ending a page

A page ends when the page builder finds its best break, or when the author asks for it.

`clearpage` ends the page and puts every waiting float on float pages. In one column it is `newpage`, a `write` to no file, an empty box and the penalty -10001 that makes the output routine run `\@doclearpage`. The write stays in the list, at the top of the next page.

LaTeX writes a float's entry for the list of figures or tables when the float's page is shipped, not when the float is read, so a float set in place before an earlier one that is still waiting is listed before it. RushTeX sets its lists from the source, before the pages exist, so when a float ships before an earlier one of its list the document goes to the engine, which lists them as pdfLaTeX does. This subsection therefore ends with a `clearpage`, which places the floats still waiting before the next subsection sets two floats where they stand.

Feature	Native
Lists	yes
Tables	yes
Floats	yes
Graphics	yes

Table T2: Feature status at the time of writing.

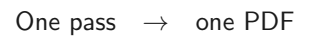


Figure 3: A starred figure.

10.4 Floats set here

The `float` package's `H` placement sets the float where it stands, between two `intextsep` skips, with its caption numbered as any other.



Figure 4: A figure placed with `H`.

Figure 4 came out where it was written, and so does Table T3, begun in the middle of this paragraph:

Table T3: A table placed with `H`.

package	papers
float	48
bm	46

10.5 Float styles and new float types

The `float` package also makes new kinds of float. `\newfloat{program}{tbp}{lop}[section]` in the preamble declares a `program` environment that floats like a figure, with its own counter numbered within the section, its own list file and the style `\floatstyle` chose last: `plain` (the caption below, as the kernel's), `plaintop` (above), `ruled` (a thick rule, the caption, a thin rule, the material and a thin rule) or `boxed` (the material framed, the caption under the frame). `\floatname` names it in its captions and `\restylefloat` gives a style to an existing kind, figures and tables among them. pdfLaTeX builds the float with `float`'s macros, its caption kept in a box of its own until the float ends; RushTeX builds the same box in one step at the float's end. With the caption package loaded, as here, the caption is caption's, in caption's `ruled` style for a ruled float: a bold label, a space and no struts. Program 10.1 is a ruled float placed with `H`, and Snippet 1 a boxed one:

Program 10.1 Euclid's algorithm for the greatest common divisor.

while $b \neq 0$ **do**
    $(a, b) \leftarrow (b, a \bmod b)$
return a

```
printf("hello, world\n");
```

snippet 1: The first program of many books.

Program 10.1 is numbered within Section 10, so its number is 10.1; the snippets are numbered on their own.

10.6 Algorithms

Pseudocode is written with three packages that work together. `algorithm` declares, through `float`, an `algorithm` float in the `ruled` style; `algorithmicx` gives the `algorithmic` environment, a list whose items are the lines, each block indented by `\algorithmicindent` and the lines numbered when the environment is given `[1]`; `algpseudocode` defines the blocks themselves: `\If`, `\ElsIf`, `\Else` and `\EndIf`, `\For`, `\ForAll`, `\While`, `\Loop` and `\Repeat` with their ends, `\Procedure` and `\Function`, and the commands `\State`, `\Statex`, `\Require`, `\Ensure`, `\Return`, `\Call` and `\Comment`. pdfLaTeX opens and closes the blocks through algorithmicx's chain of macros, one level of group per block; RushTeX keeps the open blocks in a small stack and sets the same lines. Algorithm 1, placed here with `H`, is numbered with the algorithms, on its own:

Algorithm 1 Euclid's algorithm

Require: integers $a \geq b \geq 0$
Ensure: $\gcd(a, b)$
1: **procedure** EUCLID(a, b) ▷ The g.c.d. of a and b
2: $r \leftarrow a \bmod b$
3: **while** $r \neq 0$ **do** ▷ We have the answer if r is 0
4: $a \leftarrow b$
5: $b \leftarrow r$
6: $r \leftarrow a \bmod b$
7: **end while**
8: **if** $b = 1$ **then**
9: **return** 1 ▷ coprime
10: **else if** $b > a$ **then**
11: **return** EUCLID(b, a)
12: **else**
13: **return** b
14: **end if**
15: **end procedure**

Without the `[1]` the lines carry no numbers, and an `algorithmic` needs no float around it:

for $i \leftarrow 1$ **to** n **do**
  repeat
    $x_i \leftarrow x_i/2$
  until $x_i < 1$
end for
for all $v \in V$ **do**
  visit v

```
end for

function TWICE(x)
    return 2x
end function
```

10.7 Captions

This document loads the caption package, so every caption in it, the ones above included, is caption's box and not the kernel's: the label, a colon and the text between two struts, in a paragraph box as wide as the line, centred when it fits on one line, and put above or below the float's material by where it stands (`position=auto`: what came before it in the float decides). pdfLaTeX builds that box through caption's macros on every caption; RushTeX builds the same boxes directly and keeps caption's rules for the skips around it. caption also patches float, longtable, threeparttable and listings, all loaded here: the floats placed with `H` above take the same box, and so does the listing's caption in Section 16.12, where caption swaps the two caption skips.

Outside a float, `captionof` makes a caption of a float's type, numbered with the floats; two minipages side by side are where it is used most:



Figure 5: A caption made with `captionof` in a minipage.

Table T4: Its neighbour, a table.

key	value
a	1

Figure 5 and Table T4 are numbered in sequence with the floats and listed at the front with them.

10.8 Columns

The multicol package sets text in columns inside a page. Its environment `multicols` takes the number of columns; the page above it is kept, the columns are filled to the page's height and, at the end, balanced so that they come out of equal height. RushTeX runs multicol's own algorithm: the columns are split from one box with `vsplit`, the first height tried is the material's height divided by the number of columns, and it grows a point at a time until no column is too bad and the last one is not longer than the others.

A balanced pair of columns. The text runs down the left column first and then the right one, and the environment ends with both of the same height, so that the text after it starts below the longer of the two. The space between the columns is `columnsep`, and `columnseprule` draws a line between them when it is not zero.

The second paragraph shows that a paragraph may be broken across the columns: its lines go where the balancing puts the break, and a club or widow penalty weighs on that choice as it does on a page.

Text after the environment continues in one column, after `multicolsep`.

Three columns with a rule between them. A list works inside a column:

- the first item,
- a second item that is long enough to take two lines,

and so does mathematics, $a^2 + b^2 = c^2$, set at the column's width.

The starred environment `multicols*` does not balance: its columns are filled one after the other to the page's height, the last perhaps short.

10.9 Column breaks

`\columnbreak` ends a column of `multicols` where it stands. multicol's output routine keeps what came before in a box of its own, lowers the page goal by its height and goes on; when the environment ends, the kept columns and the rest are balanced together, the break a forced one for each split.

In vertical mode it is a penalty of -10005; inside a paragraph, as here, it is

the same penalty in a `vadjust` after the line.

The rest of the text is set in the last column.

10.10 Columns in a box

Inside a box the `multicols` environment cannot cut pages: it sets its whole text in one box and balances it into columns at its end, a forced break included.

Ingredients
Two eggs, a cup of flour, a pinch of salt and enough milk to make a smooth batter that pours easily from the spoon. **Method**
Whisk, rest the batter, then cook it thinly in a hot pan.

11 Tables

11.1 Alignments and the tabular preamble

A table in TEX is an *alignment*: rows of cells, where each column has a template that wraps every cell, and glue called tabskip lies between the columns. The alignment sets every cell once, finds the widest cell of each column, and then packs all rows to the same column widths. LATEX's `tabular` environment builds the templates from its preamble, a short string of letters, listed in Table T1.

Letter	Meaning
l, c, r	a column set flush left, centred or flush right
vertical bar	a vertical rule between columns
at-sign	text in place of the space between columns
star	a pattern repeated a given number of times

Table T1: The preamble letters of `tabular` used in this document.

pdfLaTeX turns the preamble into templates with macros each time a tabular starts, building token lists character by character. RushTeX reads the preamble in Rust, and its templates are native commands, so a cell is set by the same primitives without the macro text around them.

11.2 Rules, spacing and spanning cells

Table rules come in three kinds: a full rule across every column, made with `hline`; a partial rule across some of them, `cline`; and a doubled rule made of two full rules in a row. A heading may span all the columns, and a cell may span two of them, with `multicolumn`. Table T2 in Section 20 uses all of them. This document loads the array package, which builds every table and array its own way: a vertical rule is set at its

full width between the column spaces, and a doubled rule has no backspace between its two lines.

Extra space after a row break, as in a row ended with a bracketed length, separates the groups of the next table, which lists the sizes of the records the native path moves most; its rows end with `tabularnewline`, the long name of the row break.

Record	Bytes
Token	4
Node	12
Cache line	64

A processor reads memory in lines of 64 bytes, so five nodes or sixteen tokens share a line, and a word stored as one node instead of five character nodes moves a fifth of the data through every later step.

11.3 Paragraph columns

A column of the letter p, with a width as its argument, holds a paragraph instead of a line: each cell is a top-aligned vertical box of that width, in which the text is broken into lines like any other paragraph. The array package sets a strut at the start of the first line and one at the end of the last, so the first baseline and the last depth of the cell match a row of single lines; a cell left empty still holds one line made of the two struts.

Step	What the native path does
Width	reads the argument once, when the table starts, in the font of that point
Text	breaks the cell's paragraph with the column width as the line length, and leaves the rest of the paragraph settings as a box's own
Empty	

Without the array package the same letter takes LaTeX's own definition, which has no strut in front of the first line and always backs up by one baseline before the closing strut when the cell ends outside a paragraph.

11.4 Tables of a given width

The tabularx package sets a table to a width given first, with columns of the letter X that share what the other columns leave. Its environment reads the whole table before setting any of it, sets it once with every X column as a paragraph column of the full width, measures the result, narrows the X columns by the excess shared among them, and sets it again until the width is reached; the last setting is an alignment spread to the width. The next table is set that way across the text width.

Trial	What is measured	What changes
First	the table with each X column as wide as the whole table	the excess, divided among the X columns
Second	the table with the narrowed columns	nothing, once it is within a tenth of a point of the width

RushTeX runs the trials as the package does, keeping the table's tokens once and reading them again for each trial; a table whose cells would count, label or write something twice is left to the engine.

11.5 Declarations in the preamble

The array package lets a preamble carry text of its own around each cell. A greater-than sign with a group before a column puts that group in front of every cell of the column, a less-than sign after it puts its group behind, and an exclamation mark with a group puts the group in place of a rule between columns. The next table sets its first column in bold, its second as a formula with a fixed exponent, and its third as a paragraph column whose declaration sets the text ragged right and then, with `arraybackslash`, gives the row break back to the table.

Native	x^2	the declaration's tokens are read in place, and the steps after them wait
Engine	y^2	the same tokens, written into the template by an expanded definition

The declarations are kept as token lists when the table starts; a cell's steps stop where a list belongs, the list is read as the source would be, and the steps go on after it. A group after an at-sign is read the same way, so text such as a decimal point can stand between two columns; only a column separation that changes the space between all the columns is still left to the engine.

11.6 Column types of one's own

array's `newcolumntype` names a column specification, with arguments if it has some. Before the preamble is built, every letter so defined is replaced by its text, again and again until none is left, exactly as array's `\NC@list` does. Here `P` is a ragged-right paragraph column of a given width and `Y` a ragged-left `X` column of tabularx.

Driver	Base-case assumption
Growth	Twelve per cent a year for three years, then eight, with no further change in the mix of customers
Margin	Flat

Item	Hours	Amount
Design of the column rewriting, read from array's source	6	480.00

The kernel's `tabular*` takes its width first; the space between the columns comes from `@{\extracolsep{\fill}}`, which sets the alignment's tab skip from that column on, as a résumé heading does:

Research assistant Cambridge
Typesetting group *2019 – 2021*

11.7 Columns centred and set from the bottom

array adds two more paragraph columns. In an `m` column the cell is a box centred on the row's axis, and in a `b` column a box aligned on its last line, where a `p` column aligns on its first. RushTeX sets the cell's paragraph once and wraps it in the same kind of box. A paragraph column may sit on its middle line or on its last line instead of its first:

Short	one line	last line	mid
Long	a longer text that wraps over several lines of this column	a text of two lines	centred in a narrow cell

11.8 Table rows kept in a command

pdfLaTeX reads a table's cells as tokens, so a cell boundary may come from a macro's text; RushTeX reads the same tokens and finds the same boundaries. A command of the document's own may hold the column separators of a table row: the `&` in its text is only stored when the command is defined, and becomes a column break where the command is used inside the table.

Monday	Confirm the forecast inputs
Tuesday	Interview two candidates
Wednesday	*Draft the dashboard*

11.9 Verbatim text in a table

A table's entries are read from the file one at a time, so `\verb` works in them:

Command	Level
`\part`	-1
`\chapter`	0
x `a␣b` y	`<->`

11.10 Colour in table entries

A `\color` inside an entry ends with the entry's group: its `\reset@color` comes right after the entry's text, before the column's closing glue. With colortbl (xcolor's

`table` option) a document may also colour a row with `\rowcolor`, one entry with `\cellcolor`, a column with `>{\columncolor{..}}` and the rules with `\arrayrulecolor`; this document does not load colortbl, which boxes every entry.

right	left	centre
plain	grey	also plain

11.11 Notes under a table

The threeparttable package measures the table first, then sets its notes as a list of the table's own width. pdfLaTeX does this with a box saved and measured by macros; RushTeX builds the same boxes natively, the notes' list at the table's width. A table may carry notes set to its own width, marked in the cells:

	(1)	(2)[a]
Treatment	0.12[b]	0.10
Controls	No	Yes

[a] With the controls of the second model.

[b] Significant at the five per cent level, a note long enough to wrap at the table's width.

A note with no mark.

11.12 Long tables on one page

A long table may run over many pages, which pdfLaTeX handles with a second run: the column widths found in the first run are written to the auxiliary file. RushTeX takes a long table natively when it ends on the page it starts on, and computes the widths in the one pass. A `longtable` is set in chunks, each an alignment as wide as the line; a head ended by `endhead` is a chunk of its own, and the columns of all chunks are made equally wide, as the second run of pdfLaTeX makes them from the auxiliary file. The table steps the table counter, and booktabs' rules in it are rows of leaders.

ID	Date	Decision
D01	29 August	Use the sheltered baseline as the reference run
D02	30 August	Keep the raw values

A long table that would not end on the page it starts on still runs on the engine path.

12 Pictures

Figure 6 holds two pictures included with `includegraphics`: on the left a vector drawing, a PDF page, set to a width of 55 percent of the line; on the right a raster image, a PNG file, at three tenths of its natural size. A document may load the plain `graphics` package instead: the same code without graphicx's keys, so `\includegraphics{file}`, `\scalebox`, `\rotatebox` and `\resizebox` are drawn as here, while its bounding-box brackets go to the engine until graphicx (or tikz, which loads it) comes.

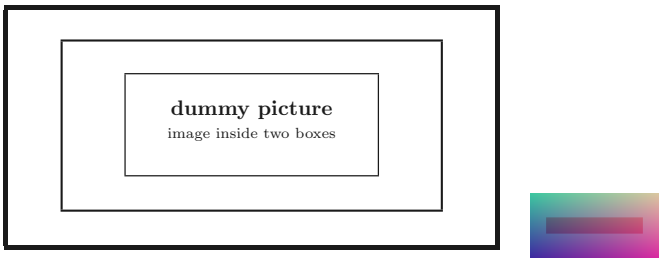


Figure 6: Pictures: a PDF page scaled to a width, and a PNG image scaled by a factor.

In pdfLaTeX the command runs four packages of macros: keyval parses the options, graphics searches for the file under each known extension, graphicx turns the options into a requested size, and the pdfTeX driver asks the engine to read the file with `pdfximage`, measures the image in a box, writes its size back as text in big points and reads that text again. The scale factors are found by a loop of doublings and divisions written in macros, and every step expands tokens and makes assignments.

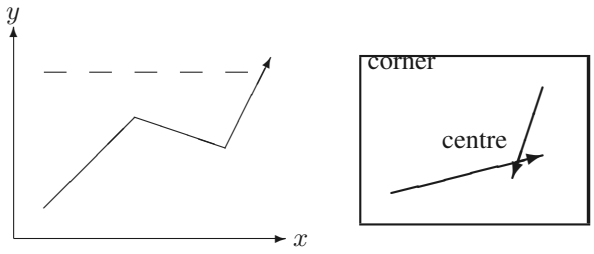
RushTeX runs one Rust function for the same work, with the same arithmetic: the size is read back through big points, the factors come from the same doublings and divisions in TeX's integer units, and a factor times a length is rounded as TeX rounds

it. The function builds the boxes the macros build, three nested boxes with the image between a save and a restore of the graphics state and a matrix that scales it, none at scale one. Only the file's header is read when the picture is placed; a PNG is decoded and a PDF page copied when the document is written, once per file however often it is shown. Loading graphicx also sets the PDF page to the paper size, as the pdfTeX driver does, so this document's pages are letter size.

Pictures drawn in the document itself are the subject of the rest of this chapter. TikZ is a library of TEX macros, pgf, that computes every coordinate in TEX's fixed-point arithmetic and writes the path as PDF operators. pdfLaTeX expands those macros for each point of each path; RushTeX computes the same numbers with the same arithmetic in Rust, and writes the same operators.

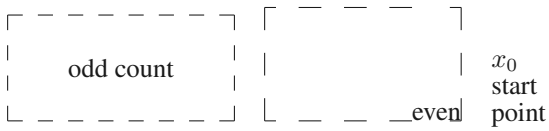
12.1 The picture environment

Before pgf, LaTeX had its own pictures, and it still has them: `picture` places objects at coordinates in multiples of `\unitlength`, and draws lines with nothing but rules and characters. A horizontal or vertical line is a rule. A sloped line is a run of short segments, characters of a font of line pieces (`line10`, or `linew10` after `\thicklines`), one for each slope whose two whole numbers are at most 6; each piece sits in its own box, raised a little more than the one before, and the last piece is placed by integer arithmetic so that the line ends where it should. Arrow heads are characters of the same font.

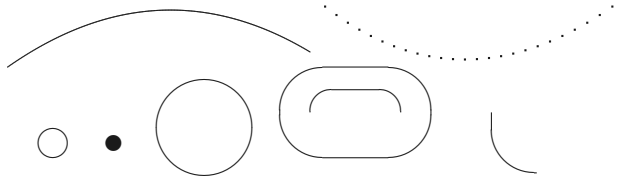


`\put`, `\multiput` and the environment itself turn into the boxes LaTeX's macros leave: each object in a box of width zero, raised and moved by its coordinates, each coordinate a number times `\unitlength` or a length of its own. RushTeX builds the line pieces itself, with the same font characters and the same arithmetic. A slope outside the font, a sloped line shorter than one piece (which LaTeX only warns about), and a circle larger than the circle font's largest quarter still leave the document to the engine.

A dashed box is a frame of short rules: `\dashbox{2}(30,14)` asks for dashes 2 units long, and LaTeX counts how many fit on each side, adds 200 scaled points of slack, and either centres an odd number of dashes or, for an even number, puts half dashes at the corners. `\shortstack` piles lines on top of each other with no line spacing but 3 points between them, centred or flushed by its `[l]` or `[r]` argument; it works in a picture, where it labels a point, and in running text, where its bottom line sits on the baseline.



Curves come from two more fonts and one loop. A circle up to 15.5 points across is a single character of `lcircle10` (filled ones too, for `\circle*`); a larger one is four quarter circles of the same font, placed by their width. `\oval` joins four quarters by rules, and its optional argument keeps only some of them: `[t]` the top half, `[bl]` the bottom left quarter. `\qbezier` draws a quadratic Bézier curve as a row of dots, each a square rule as wide as the line, as many as the curve's extent needs (at most 500) unless its optional argument says how many; the positions are LaTeX's integer arithmetic, so each dot lands on the same scaled point.



12.2 Drawings

This document loads `tikz`, and the native job loads it as pdfLaTeX does: the package and the eleven it brings (pgf's utilities, keys, mathematics, system and basic layers, `pgffor`, `graphicx` and `xcolor`) come from tables evaluated from their sources, and every page gets the box that pgf's `shipout/background` hook puts at the page origin, empty until a picture needs page-level objects.

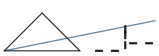
The frame around this page and the bands at its foot and corner are pictures of the kind certificates, menus and title pages draw: `remember picture,overlay`, with `draw` and `fill` of a `rectangle` between two anchors of `current page`, shifted by `xshift` and `yshift`, in a line width and a colour. pgf turns each path into `pdfliteral` operators whose numbers are TEX dimensions converted to big points by a truncating decimal multiplication, and the corners are measured from where the picture lands on the page, which pgf reads from the position it saved on the previous run; the native job writes them in the one pass that ships the page, from the same position. Pictures with nodes, other paths and other options still go to the engine.

12.3 TikZ libraries and pictures in a colour

This document also loads the `positioning` and `calc` libraries of TikZ. A library is a file of macros and keys: loading it allocates no register, box or layer, so the page is the same as without it, and the native job only checks the names. A library that does allocate, such as `arrows.meta` with its four dimensions, is native only once its registers are allocated as pdfLaTeX allocates them; this document loads it, and its tips

are drawn in Section 12.5. A picture drawn in a colour starts from that colour: pgf's `pgfsetcolor{.}` writes the current colour's values for stroking and filling, here this paragraph's colour. A gray one draws in gray.

12.4 Pictures in the text

A picture without `overlay` takes the room of its bounding box: every point of its paths, a stroked path widened by half its line width. This one is a business card's background, a rounded rectangle filled and drawn in one colour with a band across its top,  set on the line like a letter. Straight paths, corners and circles follow the same rules:  ,  .

 The numbers are TEX's: a coordinate without a unit is a multiple of the one-centimetre vectors by TEX's truncating decimal factor, a circle's control points are `0.55228475` of its radius off each end, a rounded corner's `0.5522847` of the way from where the corner is cut, and every number is printed in big points as `pgf` prints it.

12.5 Nodes

edge

A node is a line of text in a rectangle: the text box widened by the inner sep, at least the minimum size, set so that its anchor sits on the point. 

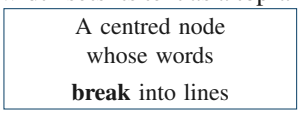
sits on the line, and so does a seal of a fixed size, 

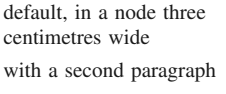
, or a filled node with no inner sep at a point, tight 

. Every size is TEX's: half of a width is `.5` of it, truncated; the inner sep is `.3333em` of the font the picture is in; the outer sep half the line width.

A node can stand on its side, 

, and in a `remember picture` picture it can sit on the page itself, here in this page's top right corner and at its right edge: pdfTEX puts such a node's text where the page's corner is less where the picture was saved, and the native job finds both as the page ships.

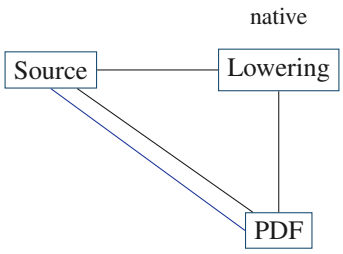
A node with a text width sets its text as a top-aligned minipage, ragged right unless `align` says otherwise: 



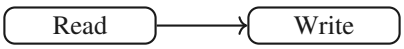
and

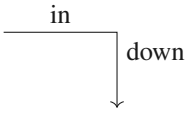
a line break in a node without one stays on its line: onetwo.

Nodes may be named, placed beside each other with the positioning keys, and joined: a line to a node ends on its border, where pgf's rectangle meets the line from the other end, and a point may be any of a node's anchors.



Without a text width, `align` sets a node's lines as an alignment, each as wide as the widest: Chapter 1 Fundamentals left aligned lines. A node's border may have rounded corners, and its text may hold \&, a bar and accented letters: R&D — café soft. A picture's options may set its line width and arrows for every path, and name a style its nodes use:





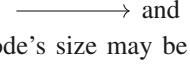
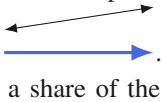
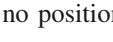
Nodes may also sit on a path, within a segment or after a point: . With xcolor's `svgnames`, the SVG colour names are known too: DarkBlue, Crimson and SeaGreen. With `dvipsnames` too, the 68 names of dvips are colours in cmyk, such as BrickRed, Periwinkle and OliveGreen; where both sets have a name, the set given later wins, as xcolor runs them in the order of its options. A node may be a circle, and lines end on its border: 1 2. Labels sit on a node's border in their direction: in gate out hot T. A picture in a colour draws its nodes in it: red on. A picture's `every node` style and the older `right of` placement: plan do

check. Mixes of cmyk colours: olive, cyan and blue, and . Stealth tips: . Curves, by control points and as a smooth plot:

over



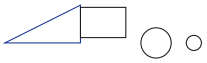
end

. Placed by a distance: under . With arrows.meta, `->` draws its Computer Modern tip, and `Latex` tips may be the picture's `>`:  and  . A picture may sit on a node's baseline, and a node's size may be a share of the text width: said in line. A circled mark is a short picture on its circle's baseline, its options followed by blanks `\tikz` skips, a plain number a length in points and a strut its text: (3) marks a step. The preamble's `\color{black!90}` makes the default colour of the whole document: every page starts in it, and `\normalcolor` (footnotes, captions, running heads) returns to it. A short `\tikz` command is a picture of one path,  , or of a braced group of commands,  , its baseline a length too (`baseline=-0.5ex` is a height of the font's x-height: ), an `overlay` picture over here without `remember picture` takes no room and marks no position on the page, and a loop may number its cells with `\pgfmathtruncatemacro`, `mod` and `int`: 1 2 3 4 5 6 7 8. In a scaled picture only the nodes' places move:  . A path's nodes take its colour and width: 

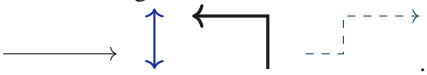


.

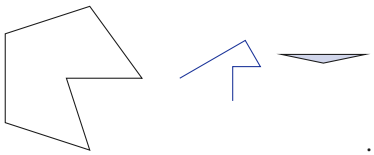
12.6 Points, paths and loops

Points may also be named, relative or lengths: a `coordinate` gives a point a name, `++` moves on from the last point and `+` measures from it without moving, and a coordinate may be a length register such as the line width. 

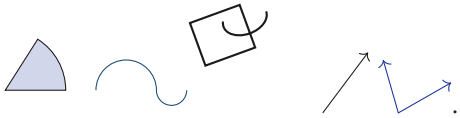
A picture may change its units and scale, and a path may shift and scale itself; line widths stay as they are:  .

Arrows end a line in `pgf`'s `to` tip, drawn back from the end by the tip's extent, which grows with the line width; the tip is drawn once per width and reused:  .

A point may also be given by an angle and a distance; the sine and cosine are `pgfmath`'s, read from its table of whole degrees and interpolated with TEX's arithmetic:



Arcs are pgf's pieces of at most ninety degrees, each a curve whose supports are `1.333333333*tan` of a quarter of its angle off the ends; paths may be turned, and an arrow on a slanted line is turned by `pgfmath`'s arc tangent of its last piece:



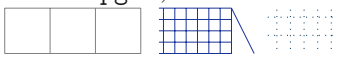
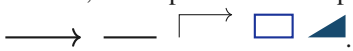
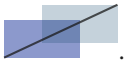
Ellipses take two radii, in either of their forms, and a path may be shifted to a point:



`\foreach` loops over literal lists, ranges with dots and pairs of values, nested or not, are unrolled as `pgffor` runs them, spaces included:  .

Coordinates may be `pgfmath` expressions of numbers, lengths and length registers, computed in pgf's fixed point:  .

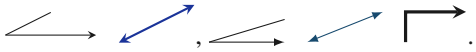
A picture may stand in a command's text, its arguments put in where the command is used:  and .

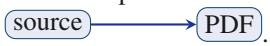
Grids are pgf's, a line at each multiple of the step and one just short of the far corner:  ; loops may count their passes:  ; named points in a scaled picture:  ; and a picture's own tips and width, for each path that strokes an open line:  . Tips take the stroke colour when the fill differs:  . Rounded corners on runs of lines, open or closed:  . Opacity goes through pgf's graphics states:  .

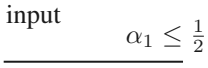


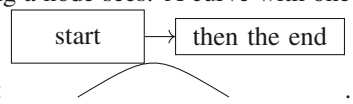
A point marked in a picture can carry a label with a formula: 

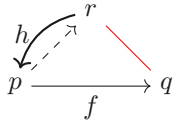
The older tips. Many documents load the `arrows` library and ask for pgf's first tips: `stealth`, a filled dart, and `latex`, a filled curved head, by name (`-latex`) or for every arrow of a picture (`>=stealth`). The library declares tips of new names and one dimension register; the tips it leaves alone are drawn here. pdfLaTeX computes each tip at its line width once, caches its drawing as one literal and pays for that first use with spaces in the picture's box; RushTeX writes the same literals and the same spaces,

so the box is the same to the scaled point:  .

Styles from the preamble. A document usually names its looks once, in the preamble: `\tikzset{rtbox/.style={...}}` or the older `\tikzstyle{rtline}=[...]`. pgfkeys stores each as a macro and typesets nothing; a picture that names the style gets its keys in place. RushTeX keeps the preamble's styles in order (a later definition of a name wins, a picture's own wins over the preamble's) and puts them in where a node or a path names them, so the pictures are drawn as with the keys written out:  .

Drawn by editors. Graphical editors such as Mathcha write their pictures in a form of their own: `x=0.75pt,y=0.75pt,yscale=-1` in the picture's options (a unit of three quarters of a point, the y axis turned down), `line width=0.75` without a unit (points, as pgfmath reads a bare number), and each label as a path of one point with a node on it, its options in several brackets: `\draw (40,30) node [anchor=north west][inner sep=0.75pt] {..` `xscale` and `yscale` are `\pgftransformxscale` and `\pgftransformyscale`, multiplying one axis of the matrix. A path of one point paints its lone moveto (nothing shows), `\path` paints nothing, and its node sits where `\node at` would put it, the point through the picture's matrix. Such documents set `\tikzset{every picture/.style={line width=0.75pt}}` in the preamble; tikz opens each picture with `\scope[every picture,`*options*`]`, so its keys go before the picture's own (not shown here: it would change every picture of this document, and a `tcolorbox` after it is left to the engine). Each bracket of a node is `\tikzset` in turn, so `[draw][red]` is `[draw,red]`. Formulas in node text may hold the kernel's symbols, fractions and sub- and superscripts:  .

Small forms of flowcharts. Node names may hold spaces, `(step 1)`, kept by tikz as written. `minimum height=2em` is read by TEX in the font around the node, not the node's own `font=`. `text centered` sets only the text's alignment in a `text width` box. `\centering` in the picture changes nothing a node sees. A curve with one support, `.. controls (a) .. (b)`, uses it twice:  .

Edges. `(a) edge[options] node {label} (b)` draws a path of its own, `\path[every edge,options] (a) --` from the last point (a node's border when it names one) to the target; the last point stays, so `(a) edge (b) edge (c)` draws two lines from `a`. tikz puts each edge, inside a scope, in the box where the path keeps its nodes, and typesets that box after the path; the path's own options are its scope, which the edges are drawn in (`\path[->, thick]` too, though it paints nothing of its own; with labels, only widths are taken there), and its labels sit midway:  .

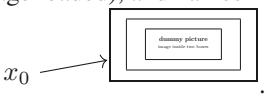
Sums of points. The calc library's `($(a)+(1,0)-(0,.5)$)` adds and subtracts points as `\pgfpointadd` does, in scaled points; a node's name stands for its centre there (its



anchor's code typeset), not for its border: t .

Scopes. `\begin{scope}[options]` opens a `\pgfscope` (a `q` in the page) whose options last to `\end{scope}`: units, shifts, scales, line widths, tips and styles. A space after its options is typeset, one after a bare `\begin{scope}` is skipped by the look for `[`, and `\end{scope}` adds its own `\endpgfidscope` space before the `Q`: ⟶ ⟶ .

Pictures in nodes. A node may hold `\includegraphics` (graphicx's, with the package loaded), and names may hold `_` and `-`; arrows to a picture end on its node's border:



.

12.7 QR codes

qrcode's `\qrcode` encodes its text as the package does in TEX: the smallest version that holds the text at the level asked, the best level that fits at that version, the Reed–Solomon blocks interleaved, the mask the package's penalty picks. Each module is a rule in a minipage, one paragraph per row:  and, at level H,  .

12.8 Coloured boxes

The `tcolorbox` package draws a framed box with a coloured interior around its text, as a picture of the native graphics layer: the frame and the interior are two filled paths with rounded corners, and the text is set in a box of the width left inside them.

A box with the package's own settings: a dark grey frame half a millimetre wide, a light grey interior, four millimetres of space at the sides and two above and below.
Its text may have several paragraphs.

A box can have a title and its own colours, given as options:

Reviewer comment

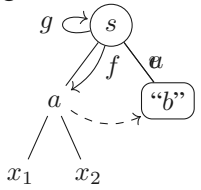
The observation window should be explained.

A box of its own

An environment defined with `newtcolorbox` gives every box the same options; its argument is the title here.

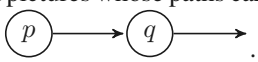
12.9 Trees in TikZ

A node followed by `child {node {..}}` groups is a tree. In pdfLaTeX tikz shifts to the parent's centre, then by the level distance down and by the sibling distance across (each child k of n at k siblings less $(n+1)/2$ of them), draws each child as a `\node` of its own in a box with the line to its parent, and puts the grandchildren in that line's box. RushTeX places the children by the same sums (a sibling distance halved as TeX halves it, so a lone child of an odd distance sits one scaled point off its parent's axis), builds the same boxes and counts the spaces the tree macros leave, one per line they end without a %. Trees take `level distance`, `sibling distance` and `level`, `level` n styles of those two keys, `grow=` in the picture's options (a direction or degrees: the level distance along it, the siblings across it), and `edge from parent` written in a child, with options (`draw=none` keeps only its spaces) and labels midway; child options still go to the engine. Node text may hold quotes, the font's ligatures (not under babel, whose shorthands may claim "). Styles may also be set inside a picture or scope with `\tikzset` or `\tikzstyle`: pgfkeys' macros, local to it, that typeset nothing. A `to` without options puts in its default path, `-- (target)` with the nodes as labels midway, so it is drawn as `--` is. With `bend left`, `bend right` or `out` and `in`, on a `to` or an `edge`, the topaths library's curve is computed as pdfLaTeX computes it: its supports at 0.3915 of the distance between the ends (by TeX's truncating divisions), turned with the line between them for a bend, the ends on the nodes' borders, a tip drawing the curve's end back along its last support, its labels where `\pgfpointcurveattime` puts them; `looseness`, `min distance` and `shorten` are taken, and an edge's `loop above` (or below, left, right) is that curve from the node back to itself, eight times as loose and at least 5mm, its tip drawn 1pt short (`every loop/.style={}` among the picture's options takes the tip and the shortening away, and `every node/.style` on a path with no nodes styles nothing; a blank line between commands is a `\par`, which does nothing in the picture: 

). A placement key in an edge's options, `edge[right] node {..}`, sets the anchor its labels start from, as `node[right]` on each would: 

.

State diagrams. The automata library's figures open with `[->,>=stealth',shorten >=1pt,auto,semithick]`. pdfLaTeX names the tip `>` in each path's tip list and makes it when the path is drawn, so `>=stealth'` after `->` still gives every edge the arrows library's rounded `stealth'` tip (its setup code, met first, leaves two pending spaces taken and five typeset); the

picture's `shorten >` draws back every open path's end, not its circles; `auto` only says how labels are anchored, so it is taken in pictures whose paths carry no labels. RushTeX draws the same tips at the same places: 

.

13 Mathematics

13.1 How a formula is set

Mathematics is set by the rules of Appendix G of *The TEXbook* [2]. A formula is first read into a list of *noads*: an ordinary symbol, an operator, a binary operation, a relation, an opening or closing delimiter, a fraction, a radical. The rules then give every noad a style, a size and a position from parameters stored in the math fonts, and insert the spaces between neighbours from a table indexed by their classes: a thin space after an operator, a medium one around a binary operation, a thick one around a relation. There are four styles, display, text, script and scriptscript, and each fraction or index moves its contents one style smaller.

pdfLaTeX expands the kernel's math macros into these primitives, and runs the rules. RushTeX ports the rules as they are and never adjusts a formula by hand; what changes is only that the math commands are Rust functions and the symbol tables, generated from the kernel and the packages at build time, are arrays.

13.2 Inline formulae

Inline formulae are written between dollar signs: $\frac{a+b}{c-d}$ is a fraction, $\sqrt{x^2+1}$ a square root, and x^2, a_i and x_i^2 show superscripts and subscripts alone and together. Greek letters are typed by name, so α, β, γ and the capital Γ are available. Relations and operations come from the symbol fonts: $a \le b$, $x \neq y$, ± 1, $u \cdot v$, $A \times B$, $A \subseteq B \cup C$, $x \in A \cap B$, $\emptyset$, $p \Rightarrow q$, $p \Leftrightarrow q$, $\forall x\, \exists y$, ∂f, ∇f and $x \mapsto x^2$.

The sum $\sum_{k=1}^{n} k$ and the integral $\int_0^1 x\,dx$ set their limits beside them in running text, as does $\lim_{n\to\infty} \frac{1}{n} = 0$. Named functions are upright: $\sin x$, $\cos x$, $\exp x$, $\log x$, $\det A$, $\ker f$, $\max_i a_i$, $\min_j b_j$, $\sup S$ and $\inf S$.

The set of real numbers is $\mathbb{R}$, and a bound is written $f(n) \in \mathcal{O}(n^2)$. A vector is bold, as $\mathbf{v}$, with length $\|\mathbf{v}\|$, a differential upright, as $\mathrm{d}x$, and a Lie algebra in Fraktur, as $\mathfrak{g}$. Delimiters grow with what they enclose: $\left(\frac{x}{y}\right)$, $\left[\frac{x}{y}\right]$, $\left|\frac{x}{y}\right|$ and $\langle x \rangle$, while $\lfloor x \rfloor$ and $\lceil x \rceil$ keep their size. The letters of a formula stay in Computer Modern while the text is in Times, because LaTeX chooses the math fonts separately from the text family.

13.3 Displays and equations

Displayed formulae are centred on their own line in the display style, with larger operators and fractions. The sum of the first n integers is

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

and the limits of the sum now sit above and below it. Inline formulae use the text style, which keeps the lines of a paragraph at an even height.

Numbered equations carry labels that the text refers to. The Gaussian integral and the quadratic formula are the next two equations.

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi} \tag{13.1}$$

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{13.2}$$

Equation 13.1 gives the Gaussian integral, and Equation 13.2 the roots of a quadratic with real coefficients a, b and c, where $a \neq 0$. A display may hold a product with a sized delimiter:

$$\prod_{i=1}^{m} \left(1 + \frac{1}{i^2}\right) \leq e^{\pi^2/6}$$

13.4 Displayed derivations

A derivation is a chain of displays, with the text between them giving the reason for each step. The square of a binomial expands as

$$(a + b)^2 = a^2 + 2ab + b^2$$

Applying such expansions to the terms of a sum gives a closed form for the sum of the squares of the first n integers:

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

The formula holds for every positive integer n and can be proved by induction, since the sum for $n+1$ is the sum for n with one more term. The constant e is the limit

$$e = \lim_{n \to \infty} \left(1 + \frac{1}{n}\right)^n$$

which the reader may compare with the bound in the product above.

13.5 Arrays of equations

Before amsmath, LaTeX's own way to align a chain of relations was `eqnarray`: three columns, the left side set right, the relation centred and the right side set left, with a

number on each line unless the line says `nonumber`:

$$\begin{aligned} (x+1)^2 &= x^2 + 2x + 1 && (13.3)\\ &\geq 2x + 1 \\ \sum_{k=0}^{n} a_k x^k &= \frac{1-x^{n+1}}{1-x}, \\ e^{i\pi} + 1 &= 0. && (13.4) \end{aligned}$$

The starred form numbers nothing:

$$f(x) = \int_0^x g(t)\,dt$$

and (13.3) and (13.4) are the lines' numbers.

The kernel builds `eqnarray` from TEX's `halign`, stretched to the display's width, with a preamble of four columns: the three parts and a box of no width for the number. Glue of one thousand points of stretch on both outer sides centres the whole. The macros around the alignment count the columns a line has reached, so that a line's end can add the missing ampersands before the number. RushTeX passes the same preamble to its engine's alignment and writes the macros as functions. A line's number is placed as the line ends, and the counter as it stands is what a label in the line records. The rows are set as they come, because unlike amsmath's `align` nothing in the layout waits for later rows. One detail matters for exactness: the brace pair that hides the next ampersand while the line's end looks for a star or a space is, in a formula, an empty group, and TEX sets it as an empty box.

13.6 Symbols built from others, and bold symbols

The kernel builds $x \notin A$ by overlaying a slash on $\in$ in each style, $A \cong B$ by stacking $\sim$ over $=$, and braces from four pieces joined by rules:

$$\underbrace{a_1 + a_2 + \cdots + a_n}_{n \text{ terms}} = \overbrace{1 + 1 + \cdots + 1}^{n}, \qquad X_{y \notin B,\ u \cong v}.$$

The `bm` package sets a bold symbol as the same character in a bold font family: $\boldsymbol{x}$, $\boldsymbol{\gamma}_k^2$, $\boldsymbol{\Gamma}$, $\mathbf{0} = \boldsymbol{\mu}$ and $\boldsymbol{\gamma}^*$, where amsmath's `boldsymbol` boxes it: $\boldsymbol{B}_x$ against $\boldsymbol{B}_x$. Here bm is loaded after amssymb and before amsmath and mathrsfs: its three families are numbered after amssymb's two, mathrsfs's after them, and amsmath's `boldsymbol`, defined later, stays its own.

13.7 Math fonts after a formula in a box

LATEX sets the math fonts globally for the size of each formula. A formula inside a box that sits in another formula (a table entry, an `\mbox` in math) puts the outer size back

after its group, so the outer formula ends with its own fonts: this table is centred on the axis of the normal size, though its entry has a tiny formula, $y = 1$, and so is z here.

13.8 Symbol and alphabet packages

Mathematical writing leans on small packages that add a font and a few names for its characters: latexsym (`\mho` ℧, `\Join` ⋈, `\Box` □, `\Diamond` ◇, `\leadsto` ⇝, ◁, ▷, ⊏), dsfont for the double-struck indicator $\mathbb{1}_A$ and the number sets $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$, bbm for the same in another cut (`\mathbb`), upgreek for upright Greek in units and constants ($5\,\mu\mathrm{m}$, π, $\Delta\lambda$ beside the italic μ, π), stmaryrd for the brackets and arrows of semantics (`\llbracket`, `\mapsfrom`), esint for the integrals of physics (`\oiint`, `\fint`), and braket for Dirac's notation:

$$|\psi\rangle = \sum_{n} \langle n|\psi\rangle\, |n\rangle\,, \qquad \langle 0|\, H\, |1\rangle\,, \qquad \mathbb{E}\big[\mathbb{1}_{\{X>t\}}\big] \le \frac{\mathbb{E}[X]}{t}, \qquad \Box\varphi \rightsquigarrow \Diamond\psi.$$

Each package takes a math family of its own, and TEX has sixteen; with these this document is at the limit the native job takes, so bbm, stmaryrd and esint are exercised in separate probes.

pdfLaTeX runs each package's macros: `\DeclareSymbolFont` gives the font the next free family, `\DeclareMathSymbol` makes each name a `\mathchardef`, and the first use of a size loads the font's `.fd` file. RushTeX reads none of that at run time. Each package is a table that `rushtex-gen` took from pdfLaTeX itself after loading it: its symbol fonts and alphabets, its characters with their class, family and slot, and its macros as tokens. Loading the package installs the table, moving its families up past those declared before it, as `\DeclareSymbolFont` would; the `.fd` shapes and the fonts' metrics are carried in the binary. A name the table could not hold (stmaryrd's `\oast`) sends the document to the engine.

braket's set commands are mechanisms, not data: inside the argument the bar is made math-active, and the active bar closes the group it is in, puts a divider and opens a new one. In `\set` and `\Set` that first divider ends the group that made the bar active, so later bars are ordinary; in `\Braket` every bar divides, and a double bar gives a double divider grown with `\middle`:

$$\{\, x \in \mathbb{R} \mid x^2 < 2 \,\}\,, \qquad \left\{ \frac{1}{n} \,\middle|\, n \in \mathbb{N} \right\}, \qquad \left\langle \phi \,\middle|\, \frac{\partial^2}{\partial t^2} \,\middle|\, \psi \right\rangle, \qquad \{\, a \parallel b \,\}\,.$$

RushTeX splits the argument at its top-level bars itself and passes the pieces on as groups with the divider between, the token list braket's macros leave. A bar the active one would reach some other way (inside a group, after a command that may read it such as `\left`, or in a macro's text) leaves the document to the engine.

13.9 Fractions with a slash

The nicefrac package sets a fraction as a raised numerator, a slash and a lowered denominator, for text that a built-up fraction would crowd: ½ inch, 3/16 of the width, and

in a formula N/B, $\lceil n/2 \rceil$, $x^{1/2}$ or

$$y = 3/4\, x + \pi/2.$$

In text the numerator and denominator are in the script size of the current font, and the numerator is raised by the height of an M less the height of a small M; in a formula a `\mathchoice` picks the script style (scriptscript in a script) and a raise from the heights of M in two of the math fonts' sizes. pdfLaTeX runs the macro, measuring each M with `\settoheight` in a box of its own every time. RushTeX reads the heights from the fonts' metrics and passes the arguments on to the same boxes, kerns and slash; units' `\unit` and `\unitfrac` are built the same way (upright in a formula) and are exercised in a separate probe, since units and siunitx both define `\unit`.

13.10 Cancelled terms

The cancel package strikes terms out. Its slash is a line of the `picture` environment (section 12.1): the term is set in a box and centred on the axis, and the package picks the slope from the box's shape, steep for a tall term and shallow for a wide one, and the length from its width or height, in a few cases of integer arithmetic. `\bcancel` draws the slash falling, `\xcancel` both, and `\cancelto` draws an arrow with the value the term tends to at its head, one style smaller:

$$\frac{\cancel{(x+1)}(x-1)}{\cancel{(x+1)}} = x - 1, \qquad \cancel{\sum_{k=1}^{n} k + \xcancel{k}}, \qquad \cancel{\frac{\partial^2 a}{\partial t^2}} + \frac{1}{\cancelto{0}{\epsilon}}.$$

In text the term is a box of its own: ~~word~~ and ~~two words~~. RushTeX measures the box with the engine's own `\setbox`, does the package's arithmetic in Rust, and draws the line with the native picture `\line` and `\vector`.

The package's options are native too. `makeroom` gives the slash and the arrow their own width (`\hfil` instead of `\hidewidth` around them), so that a value wider than its term pushes the next symbol aside instead of overlapping it; `samesize` and `Smaller` set the value of `\cancelto` in the term's own style or two styles smaller, and `thicklines` draws with the thick line font. LaTeX runs a package's options in the order the package declares them, not the order they are given, so `[makeroom,overlap]` and `[overlap,makeroom]` both overlap.

Many documents load a package and never use it. This one loads xfrac: its table holds what loading it changes (`graphicx` and `amstext`, its template commands), and leaves `\sfrac` to the engine, whose scaled fonts (xfrac's instances) are not ported yet. A document that uses it runs on the engine; one that only loads it stays native. adjustbox is the same: loading it loads graphicx and six helper packages (xkeyval, adjcalc, collectbox, trimclip, ifoddpage, varwidth), whose tables RushTeX holds; one of them, ifoddpage, writes a line to the `.aux` file as it loads, which leaves a write node on the first page, and RushTeX leaves the same node. `\adjustbox` and its keys still go to the engine.

14 The amsmath package

14.1 What amsmath changes

amsmath is the package most mathematical papers load. It changes more than it adds: the equation number becomes a tag set by its own macros, the size of hand-sized delimiters follows the font, the thin space and the colon get new definitions, and accents and dots inspect what follows them. A paper that loads it is compiled natively only if every such change is followed exactly, so RushTeX's generator sorts each name amsmath changes into four kinds: a definition it can carry as data, a native command that follows amsmath once it is loaded, a change that leaves the page unchanged, and a name that sends the document to the engine path. Section 16 describes the generator.

14.2 Text, operators, dots and delimiters

Words inside formulae are set in text boxes: $f(x) = 0$ for all $x \in \mathbb{R}$, and in an index x_{max} the words shrink with the index. Text may hold a formula with text of its own, for $x \in$ A only: each of the four copies TEX sets of a text box knows its own style. Operator names are upright with their own spacing, as in $\operatorname{tr} A$, declared in the preamble with `DeclareMathOperator` (also over a name first let to `relax`, as $T_n = \operatorname{o_P}(1)$ is), $\operatorname{rank} M$ and

$$\hat{\theta} = \arg\max_{\theta} \log L(\theta), \qquad \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i,$$

where the starred form puts the index below the name in a display.

Dots choose their height from their neighbours: $x_1, x_2, \ldots, x_n$ sit on the baseline between commas and $x_1 + x_2 + \cdots + x_n$ on the axis between plus signs. The author may also choose: $x_1 \cdots x_n$ for centred dots, $a_1, \ldots, a_n$ for dots between commas and $a_1 + \cdots + a_n$ for dots between binary operations. Wide accents stretch over their argument, as in $\widehat{xyz}$ and $\widetilde{abc}$. A map is written $f\colon A \to B$.

Sized delimiters are written by hand when the automatic ones are too large or too small. The four sizes, `big`, `Big`, `bigg` and `Bigg`, each come in an opening, a closing and a middle form: $\big(x\big)$, $\Big[y\Big]$, $\bigg\{ z \bigg\}$ and $\Big| w \Big|$. In a display:

$$\Big(\sum_{k} a_k \Big)^2 \le n \sum_{k} a_k^2 .$$

amsmath computes their size from the height of a parenthesis in the current font, so RushTeX's native command takes it from the same metric.

14.3 Equations and their tags

A numbered equation of amsmath sets its tag after the formula:

$$e^{i\pi} + 1 = 0 \tag{14.1}$$

and the `eqref` command prints the reference with its parentheses, so the text reads (14.1). A display may also go unnumbered with the starred form,

$$\vec{F} = m\ddot{x},$$

which is the same as the bracket form used in Section 13. An equation whose number is suppressed with `notag` also has none:

$$\exp(i\theta) = \cos\theta + i\sin\theta.$$

14.4 Aligned equations in one pass

Several equations aligned at their relation signs use `align`, which numbers each row unless told otherwise:

$$(a+b)^2 = a^2 + 2ab + b^2 \tag{14.2}$$

$$(a-b)^2 = a^2 - 2ab + b^2$$

$$(a+b)(a-b) = a^2 - b^2 \tag{14.3}$$

so (14.2) and (14.3) carry numbers and the middle row none. The starred form numbers no row, and pairs of columns set several equations side by side:

$$x = r\cos\theta, \qquad y = r\sin\theta,$$

$$r^2 = x^2 + y^2, \qquad \tan\theta = y/x.$$

amsmath sets such a display twice: a first pass measures every column, row and tag, and a second sets the rows with the space between the columns that the first pass found. RushTeX sets it once. The number of columns is counted from the text before the display starts, the rows record their widths as they are built, and at the end the same arithmetic fixes the space before, between and after the columns and places each number; the rows already built are then given those values. The second pass of amsmath typesets every formula of the display again, the costliest work an alignment does, and RushTeX does not do it at all.

A row may carry its own tag in place of a number, and the rows after it count on:

$$u = v \tag{14.4}$$

$$w = z \tag{T}$$

$$s = t \tag{14.5}$$

so (14.4), (T) and (14.5) are the labels of these rows.

14.5 Gathered equations

Equations that belong together but share no relation sign to align on are gathered: each row is centred on its own, and numbered unless told otherwise.

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0} \tag{14.6}$$

$$\nabla \cdot \mathbf{B} = 0$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \tag{14.7}$$

Gauss's law (14.6) and Faraday's law (14.7) are numbered, the middle row is not. The starred form numbers no row:

$$a + b = c$$

$$x^2 + y^2 + z^2 = r^2$$

A gathered row moves left of centre only when its number would otherwise crowd it: its shift depends on its own formula and its own number, nothing else. amsmath still sets the whole display twice to learn each number's width. RushTeX sets each row once and writes its shift in when the display ends, once every number is known.

14.6 Equations too long for a line

An equation too long for one line is broken with `multline`: the first row sits against the left margin, the last against the right, the rows between are centred, and one number goes with the last row.

$$\begin{aligned}(x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8)^2 = {} & x_1^2 + x_2^2 + x_3^2 \\ & + x_4^2 + x_5^2 + x_6^2 + x_7^2 + x_8^2 \\ & + 2 \sum_{1 \le i < j \le 8} x_i x_j\end{aligned} \tag{14.8}$$

Equation (14.8) has three rows and one number. amsmath measures the body first, to learn the width of the last row and so whether the number fits beside it; RushTeX learns that width as the last row is set, just before the number is placed.

14.7 Aligned blocks inside a formula

A block of aligned rows can also be one piece of a larger formula, with a single number for the whole: `aligned` is centred on the formula's axis, beside whatever else the formula holds.

$$\begin{aligned}(a + b)^3 &= (a + b)(a + b)^2 \\ &= a^3 + 3a^2 b + 3ab^2 + b^3\end{aligned} \tag{14.9}$$

Equation (14.9) has two rows and one number. Its sibling `gathered` centres each row, and either can sit at the top or the bottom of its line instead:

$$\begin{gathered} x = p = q \\ r = s \end{gathered} \qquad \begin{gathered} a + b \\ c \end{gathered}$$

Neither measures anything first: amsmath sets their rows once too, so RushTeX simply follows it, row by row. One may sit in a field of another; the inner one does not open its rows up a second time:

$$\begin{aligned} E = {} & \exp(a) \\ & \cdot \exp(b) \\ = {} & \exp(a + b) \end{aligned}$$

14.8 One equation over several lines

When a single equation needs several lines aligned at a sign, `split` inside `equation` keeps one number for all of them, centred on the rows:

$$\begin{aligned} \sum_{k=1}^{n} (2k - 1) &= 2 \sum_{k=1}^{n} k - n \\ &= n(n + 1) - n \\ &= n^2 \end{aligned} \tag{14.10}$$

so the odd numbers add up to squares, as (14.10) shows. A fraction in a line of text is set small, $\frac{1}{n}$, unless asked to be full size, $\dfrac{1}{n}$; in a display the reverse holds, and $\tfrac{1}{2}$ asks for the small one. Binomial coefficients, such as $\binom{n}{k}$ in text or

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!},$$

are fractions without a bar between brackets, all made by amsmath's `genfrac`. An equation may also carry a tag of its own instead of a number:

$$e^{i\pi} + 1 = 0 \tag{E}$$

and (E) refers to it by that tag. Script letters come from mathrsfs: $\mathscr{L}(\mathscr{H})$ beside $\mathcal{L}(\mathcal{H})$. Equations here are numbered within their section, as amsmath's `numberwithin` asks: (13.1) is the first of its section. Long arrows join a bar to a head, as in $f \colon A \longrightarrow B$ or $p \iff q$, and spacing commands fine-tune a formula: $a\,b$, $a\ b$, $a\ b$ and $a\!b$. A smashed or phantom box keeps only some of its size: $\sqrt{y}$ against $\sqrt{\smash{y}}$, or $\sqrt{a}$ beside $\sqrt{}$. Residues are written $a \equiv b \pmod{m}$ or $a \bmod m$, and text symbols such as \, |, § 2, § 3 and a dagger† need no math mode. A symbol can carry another above or below it, as in $a \overset{\text{def}}{=} b$ or $\lim x_n \underset{n \to \infty}{\longrightarrow} x$, and a matrix can elide rows and diagonals with $\vdots$ and

$\ddots$ $\vdots$

$$\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} \quad \text{or} \quad \begin{bmatrix} b_1 & c_1 & \dots & 0 \\ a_2 & b_2 & \dots & \vdots \\ 0 & \dots & a_k & b_k \end{bmatrix}.$$

A plain `\dots` at the end of an entry has no neighbour to follow, so it sits on the baseline. Vectors and matrices are often set in bold: $\boldsymbol{Ax} = \boldsymbol{b}$ with $\boldsymbol{\alpha} + \boldsymbol{\beta_k}$ takes amsmath's `boldsymbol`, and $\boldsymbol{x^2} + \boldsymbol{\theta} \leq \sum_{\boldsymbol{i}} \boldsymbol{y_i}$ the whole formula in the bold math version. Conditions stack under a sum with `substack`, as in $\sum_{\substack{0 \leq i \leq m \\ 0 < j < n}} P(i,j)$, and an arrow stretches to its labels: $A \xrightarrow{f} B \xleftarrow[\text{below}]{g \circ h} C$. Arrows over a symbol take its width: $\overrightarrow{AB} + \overleftarrow{CD} = \overleftrightarrow{EF}$, also in a script, $a^{\vec{v}}$. Roots of any order take an index, $\sqrt[3]{x}$ or $\sqrt[n]{1 + y^2}$, and the plain commands still work after amsmath, which only warns in the log: $\frac{a}{b}$, $\binom{n}{k}$, $x \stackrel{\text{def}}{=} y$ and $a \doteq b$. amsmath builds this as a gathered display of one row, whose only entry is the split's block of rows; RushTeX builds the same boxes and sets the number once the block is complete.

14.9 Cases, matrices and arrays

A definition by cases, a matrix and a determinant are all one construction: LaTeX's `array`, a table whose every entry is a formula. amsmath writes `cases` as an array of two left-aligned columns with a quad between them inside a left brace,

$$|x| = \begin{cases} x, & \text{if } x \geq 0, \\ -x, & \text{otherwise,} \end{cases} \tag{14.11}$$

and each matrix environment as an array of ten centred columns between its own delimiters, with the space outside the first and last columns taken away:

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{bmatrix} x \\ y \end{bmatrix}, \qquad \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc, \qquad \left\| u \right\|, \quad \left\{ 1 \quad \cdots \quad n \right\}, \quad p \quad q.$$

A plain matrix between `\left[` and `\right]` may space its rows too: $\left[\begin{smallmatrix} 1 & \frac{5}{4} \\ 0 & \frac{7}{4} \end{smallmatrix}\right]$. A plain array takes any column pattern, rules and a space after a row, as in $\left(\begin{array}{c|cc} 1 & 2 & 3 \\ \hline x & y & z \end{array}\right)$, and an array may sit in an aligned row:

$$f(x) = \begin{cases} 0 & \text{if } x < 0, \\ x^2 & \text{if } 0 \leq x \leq 1, \\ 1 & \text{otherwise,} \end{cases} \tag{14.12}$$

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}. \tag{14.13}$$

So (14.11) and (14.12) are numbered as any other display. A small matrix fits in a line of text, its entries in script size: $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$. An entry may span columns, with `multicolumn` or with a row of dots from `hdotsfor`:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ \cdots & \cdots & \cdots \\ a_{31} & \cdots & \cdots \end{pmatrix} \qquad \begin{matrix} 1 & 2 & 3 \\ x+y & & z \end{matrix}$$

In LaTeX the array's column pattern becomes text, the *preamble*, which is read back as tokens at every entry, and each entry opens and closes its own formula. RushTeX reads the pattern once into the same list of steps it uses for a `tabular` (Section 11), with three more kinds of step: the formula's opening and closing dollar signs and amsmath's quad. One mechanism serves tables, arrays, the cases and the six matrices, and the details that decide the output are kept: the empty formula LaTeX leaves at the end of each row, the formula that an empty at-sign argument makes, and the extra thin space amsmath's dots take before an ampersand, where its look-ahead meets the dollar sign of the next entry.

14.10 mathtools and calc

This book loads `mathtools`, which loads `calc` and amsmath, and graphicx at `begin{document}`. Its `overbrace` and `underbrace` kern 0.7 and 0.5 of the axis height (the fifth parameter of the symbol font) where the kernel kerns 3pt, so the braces of Section 13.6 sit closer to their arguments. Its `nonumber`, `notag` and ends of the align environments do what amsmath's do when `showonlyrefs` is off. With calc, `setlength` and `setcounter` take a plain value natively; an expression such as `\textwidth-2cm` sends the document to the engine.

mathtools adds cases environments of its own. `dcases` sets both columns in display style, `rcases` puts the brace on the right, `drcases` does both, and a starred form (`dcases*`, `rcases*`, `cases*`) takes its second column as text:

$$\sigma(x) = \begin{cases} \dfrac{1}{1+e^{-x}} & x \ge 0, \\[2ex] \dfrac{e^{x}}{1+e^{x}} & x < 0, \end{cases} \qquad \left.\begin{matrix} a = b \\ c = d \end{matrix}\right\} \Rightarrow e, \qquad n = \begin{cases} \dfrac{m}{2} & \text{when } m \text{ is even} \\[2ex] 3m+1 & \text{otherwise.} \end{cases}$$

They are not amsmath's array: mathtools builds a `vcenter` holding an `ialign` of two columns, a quad between them, each entry starting with amsmath's strut, and spreads the rows by `jot` unless an enclosing display already has. RushTeX writes that alignment once from the environment's name and runs it on its own alignment code; the spreading follows amsmath's rule that it happens once per group.

A starred matrix of mathtools takes the alignment of its columns as an optional argument, as in $\begin{pmatrix} -1 & 2 \\ 3 & -40 \end{pmatrix}$ or $\begin{bmatrix} x & -y \\ -10 & z \end{bmatrix}$; it is amsmath's matrix with `l` or `r` in place of `c` in the array's column pattern, and RushTeX gives that pattern to the same array code. mathtools also adds extensible arrows to amsmath's two: $x \xmapsto{\varphi} y$ and

$A \xleftrightarrow[\text{iso}]{g} B$ are amsmath's `ext@arrow` with another arrow fill and other kerns around the labels, and are built by the same code as $\xrightarrow{f}$.

A delimiter pair may also take arguments and a body: this book declares `rtset` with `DeclarePairedDelimiterX` as braces around two arguments separated by a bar of the same size, `delimsize\vert`, so $\{x \in X \mid x > 0\}$ has plain braces, $\left\{a \;\middle|\; b\right\}$ big ones and $\left\{x \;\middle|\; \frac{x}{2} > 0\right\}$ braces and a bar that grow with the contents. mathtools writes three macros for each such pair and a fourth for the body; RushTeX keeps the body as the document wrote it, puts the arguments in its parameters when the pair is used, and sets `delimsize` for that use alone.

amsmath's `sideset` puts symbols at the four corners of a large operator without moving its limits, as in

$$\sum_{n<k,\ n\ \text{odd}}' nE_n \qquad \text{and} \qquad \prod_{a\,i=1}^{b\,N}{}_{c}^{d}\, x_i.$$

amsmath measures the operator with the left symbols on an empty box of its height and with the right ones attached as scripts, then backs up by the difference so that the limits stay centred on the operator. These are assignments to box registers 0 to 6 and to `dimen0`, which a document can read afterwards; RushTeX makes the same assignments in the same order rather than computing the result in another way.

14.11 Display breaks

amsmath sets `interdisplaylinepenalty` to 10000, so its multi-line displays never break across pages. `allowdisplaybreaks[n]` in the preamble sets it to 10000, 9999, 6999, 2999 or 0 for n from 0 to 4 (4 when n is left out); this book uses 1, so a break between rows is allowed but only as a last resort. The native path reads that value at every row end of `align`, `gather`, `multline` and, under amsmath, `eqnarray`. In the body, where it is a local assignment, the command still sends the document to the engine.

14.12 Tags made of formulas and commands

A tag need not be plain text. amsmath keeps the argument of `tag` as tokens: it becomes the label's value and is set in the tag's box, between parentheses unless the tag is starred, so a formula, a font command or a reference may stand there:

$$\Phi(x) = \int_{-\infty}^{x} \varphi(t)\,dt \tag{$*$}$$

$$p = q \tag{14.11′}$$

$$r = s \tag{† **B**}$$

and (∗) and † **B** print the same tokens again. pdfLaTeX expands the stored tokens at every use; RushTeX does the same with the tokens it read, while a tag of plain characters is still built directly as text. Only a reference made before such a tag is not native: the pre-scan, which predicts every label before the body runs, cannot predict how the tokens will print, so that document goes to the engine.

15 Theorems

LaTeX's `newtheorem` makes a new environment and a counter for it. Written `newtheorem{theorem}{Theorem}[secti` the counter is reset by each section and printed after the section's number; written with a bracket before the title, as this document's lemmas are, a new environment shares an existing counter. The amsthm package, which this document loads, adds three styles chosen by `theoremstyle`, a starred form without a number, and the `proof` environment.

Definition 1 (Lowering)**.** A *lowering* of a command is a function that leaves the same boxes, glue and penalties as the command's macros, without expanding them.

Theorem 15.1. *If every command of a document has a lowering, and the guards accept the document, its native output is the output of the engine, box for box.*

Proof. Each lowering leaves what its command's macros leave, and the guards send every other document to the engine. So the two paths build the same lists, and the same lists make the same pages. □

Lemma 15.2 (Counters)**.** *A lowering of `newtheorem` needs only the environment's name, its title, its style and the counter it steps:*

$$n_{\mathrm{thm}} = n_{\mathrm{thm}} + 1, \qquad \mathrm{label} = \mathrm{section}.n_{\mathrm{thm}}.$$

Proof of Lemma 15.2. The label is the section's number and the counter's, and nothing else in the head depends on the document. □

Remark. A remark has an italic head, an upright body and half the space around it.

So Definition 1, Theorem 15.1 and Lemma 15.2 are numbered as LaTeX numbers them, in the single pass.

In the kernel a theorem is a `trivlist` with one item: the bold title and number are the item's label, and the body is italic. amsthm instead sets the head in a box that waits for the first paragraph, puts it in that paragraph's first line, and adds the period and a stretchable space after it. A proof is a list item labelled *Proof*, and it ends with an open box pushed to the right margin. Both go through many macros: `@thm`, `@begintheorem`, `deferred@thm@head`, `thmhead`, `pushQED`, `popQED` and `qed`. RushTeX keeps each theorem's title, style and counter in a small table, steps the counter itself, and builds the finished head once. The vertical space above and below, the head's place in the line and the end mark come from the same list code as every other list. One detail matters for exactness. The number is set with `textup`, whose look-ahead meets the macro that gives the period rather than the period itself, so a zero italic correction stays before the period.

15.1 Theorems within theorems, titles with commands

`newtheorem{corollary}{Corollary}[theorem]` resets the corollary counter with each step of the theorem counter (a lemma sharing it steps it too) and numbers it `\thetheorem.\arabic{corollary}`. Since `\@stpelt` steps a reset counter from -1, a new section resets the corollaries along with the theorems.

Theorem 15.3. *A theorem.*

Corollary 15.3.1. *Its first corollary.*

Lemma 15.4. *A lemma, which steps the shared counter.*

Corollary 15.4.1. *A corollary numbered again from one.*

EXAMPLE 1. *A title may hold font commands; this one is set in small capitals.*

The references read 15.3, 15.3.1 and 15.4.1.

15.2 The end mark inside a display

A proof that ends with a display would put its open box on an empty line below it; `qedhere` moves the box into the display. amsthm decides where by the name of the environment it meets. In `equation*` the box becomes the display's equation number. In a numbered `equation` the number and the box are stacked at the right margin, the number above:

Proof. The energy of a body at rest is its mass times the square of the speed of light,

$$E_0 = mc^2. \tag{15.1}$$

□

In `align*` the row that holds `qedhere` ends with amsthm's own row end, which puts the box at the right margin as a gathered display places its tag:

Proof. By the triangle inequality, term by term,

$$\begin{aligned} |a+b+c| &\le |a+b| + |c| \\ &\le |a| + |b| + |c|. \end{aligned}$$

□

pdfLaTeX sets an `align*` twice, and the box takes part in both passes: as a tag in the measuring pass, so the columns make room for it, and as the row's end in the second. RushTeX sets the rows once, counts the box's width as the measuring pass does, and writes the row's last entry when the column widths are known, as it does for tags. Equation (15.1) keeps its number.

16 Packages and their options

16.1 How a package becomes native

A package is a file of macros, and pdfLaTeX reads and expands it in every run of every document that loads it. RushTeX's generator, `rushtex-gen`, runs each package once, at build time, on the engine path, and compares the state of TEX before and after: every parameter, every character and category code, and the meaning of every command name. What the package added is sorted into tables: math symbols, operators, delimiters, alphabets, font shapes, and macros whose text can be carried as data. A name whose behaviour cannot be carried is recorded as blocked, and a document that uses it goes to the engine path. The tables are checked against the package's own result before they are written.

The gain is that loading a native package costs a table lookup, not the reading of its file, and that every number it sets is the number the package itself would have set.

16.2 Package options

A package may be loaded with options, and the preamble of this document gives every package it loads a few: `utf8` for inputenc, `psamsfonts` for amssymb, `pdftex` and `final` for graphicx, and `centertags`, `sumlimits`, `nointlimits` and `namelimits` for amsmath. Each of these asks for what the package would do anyway: UTF-8 is LATEX's own input encoding, the PostScript versions of the AMS fonts are the fonts TEX Live installs, pdfTeX is the driver graphicx would choose, and amsmath already puts limits on sums and beside integrals. So the page is the same, $\sum_{k=1}^{n} k$ and $\int_0^1 x\,dx$ inline as well as

$$\sum_{k=1}^{n} \frac{1}{k^2} \le 1 + \int_1^n \frac{dx}{x^2}, \qquad \lim_{n\to\infty} a_n.$$

RushTeX does not run a package's option code to find this out. Its generator loads each package twice, once with the option and once without, and compares the whole state afterwards. When nothing differs except the record of which options were given, the option is entered in a table of options with no effect, and a document whose options are all in that table stays on the native path. An option that does change something, such as amsmath's `fleqn`, which moves displays to the left, or `intlimits`, sends the document to the engine path until a native for it exists.

16.3 Packages loaded and not used

Lab report templates often load siunitx and never set a quantity with it. Loading it is not nothing: the package loads amstext and array as it starts, and color when the document begins, so the tables follow array's rules and every page's head and foot sit in a colour group, as they do under xcolor (Section 16.5). RushTeX loads those three in its place. None of siunitx's own commands is native yet, so a quantity, a number or an `S` column sends the document to the engine (`scripts/native_probes/siunitx-load.tex`).

16.4 Page layout

The page of this document is set by the geometry package, with the numbers the article class would have chosen itself: a text block 345pt wide and 550pt high, its left edge 62pt and its top 16pt past TEX's one-inch origin, given in scaled points so that no decimal rounds. geometry finds the right and bottom margins from the paper size, and a second `geometry` command in the preamble keeps the keys given before it and adds its own. The page therefore looks as it would without the package, and any slip in the arithmetic moves every line of the document.

16.5 Colour

This document loads the xcolor package and, apart from these words and these, sets no colour, yet the package changes every page: once a colour package is loaded, LATEX's `normalcolor` pushes the default colour, black, onto pdfTEX's colour stack and pops it at the end of the group. That happens around each page's head and foot, around every float, inside each caption, list label and theorem head, at the footnote rule, in the numbers of `eqnarray` and in the page numbers of the contents. The marks change nothing on the page; they only have to be where LATEX puts them. A colour given by name, by a model's values, or defined in the preamble with `definecolor` becomes the literal pdfTEX writes, digit for digit as the colour packages print it.

16.6 Mixing printing colours

A colour of the printing model mixed with another is kept in that model, the other colour converted to it first: cyan with blue, magenta with black, gray with cyan and cyan with orange, where the black of orange is taken out of its three inks.

16.7 Web addresses

The url package sets an address such as `https://example.org/rushtex/a_b-c?page=1&x=2#top` in the typewriter font as one formula, so that it can break after a slash, a dot or another punctuation mark but never inside a word, and so that none of its characters has its usual meaning: `~user/50%off` keeps its tilde and its percent sign. A file name is given with `path`, as in `/usr/local/texlive`; example.org is the same address in the roman style.

Under the hyperref package an address becomes a link, and `href` links a text of the writer's own to an address. hyperref reads that address with the special characters made ordinary, as url does, when it comes straight from the file. Inside the argument of another command, a table cell or a group read whole, the address has been read already, for pdfLaTeX too, so its letters and other characters are the address as written; one holding a command, a space or a special character sends the document to the engine (`scripts/native_probes/hyperref-href-argument.tex`).

hyperref also gives each heading an entry in the PDF's outline, the bookmarks a viewer lists beside the page. The title is written as `pdfstringdef` makes it: two bytes to a character (UTF-16), each byte an octal escape unless it is the low byte

of a printable ASCII character other than a space, a parenthesis or one of `#$%&_`. A title may therefore hold accented and other non-ASCII letters typed as UTF-8, dashes and curly quotes, and `\&` and its four kin, and its bytes equal pdfLaTeX's (`scripts/native_probes/hyperref-bookmark-utf8.tex`).

16.8 Quantities and units

siunitx parses each number and unit with expl3 macros, deciding at every character what to print. RushTeX parses them once, with the package's default settings, and builds the same math list. siunitx prints numbers and units in math, at its default settings: a speed of $0.42\,\mathrm{kW\,h}$ per day, 1250 samples, an acceleration in $\mathrm{m\,s^{-2}}$, a resistance of $0.12\,\Omega$ that changes by $\Omega\,^{\circ}\mathrm{C^{-1}}$, a volume of $200\,\mathrm{mL}$ at $86.0\,^{\circ}\mathrm{C}$. In a formula the unit joins it, $v = 3\,\mathrm{m\,s^{-1}}$, and a script may be a math alphabet, as in T_{room}.

Angles and ranges follow the same defaults. `\ang{12.5}` is the quantity of the number and `\degree`, 12.5°, and `\degree`, `\arcminute` and `\arcsecond` alone as a unit take no thin space before them, 90° and $3'$ against $3\,^{\circ}\,\mathrm{s^{-1}}$. A range prints each bound as a formula of its own with the phrase "to" between, in an `\mbox`: 10 to $12\,345$ samples at speeds of $0\,\mathrm{m\,s^{-1}}$ to $3.6\,\mathrm{m\,s^{-1}}$ or $1.5\,\mathrm{kg}$ to $20\,\mathrm{kg}$. siunitx reaches each of these through its expl3 key system and the translations package; RushTeX builds the noads and the phrase directly, and leaves a range in a formula or in another language to the engine.

An `S` column with a `table-format` aligns its numbers on the decimal marker, the integer part in the width of the format's digits and the decimal part followed by the space its missing digits leave; a braced entry is text, centred.

Run	Input (V)	T (s)
1	1.00	0.084
2	20.5	1.2
3	3	2.009

16.9 Degree and Celsius signs without siunitx

Lab reports that do not load siunitx often load gensymb for `\degree`, `\celsius` and `\ohm`. The package decides what they mean only at `\begin{document}`: when textcomp is loaded they are the text companion symbols `\textdegree`, `\textcelsius` and `\textohm` (and `\micro` is `\textmu`), with a new symbol font for their use in a formula; otherwise `\degree` is `\ensuremath{^\circ}`, a superscript on the atom before it or on an empty one, and `\celsius` a degree sign and an upright C. pdfLaTeX runs that hook and the macros it defines. RushTeX makes the same choice when the document starts and builds the result directly: the text symbol in the current font, or the superscript and the character as math noads. In a formula with textcomp loaded the new font would be a math family of its own, which RushTeX leaves to the engine; this document loads textcomp and is at the sixteen-family limit, so gensymb is exercised in separate probes.

16.10 Numbers wider than their column format

siunitx keeps the space of its `table-format` for each part of a number in an `S` column. A part with more digits than the format still prints: the integer part is set to the format's width behind an `\hss`, so it sticks out to the left, and the decimal part pushes the space after it below zero. A percentage is a unit like the others, $45\,\%$ and $\%$: siunitx declares it as the character `%`.

Force	Length
100	0.42
2	12.125
30.5	1.3

16.11 Units written out and long numbers

A unit need not be built from siunitx's unit macros: written out as letters, it is set upright as it is, a dot between two units stands for the thin space of a product, and a power goes after its unit. So a concentration of $0.049\,14\,\mathrm{mol\,L^{-1}}$ reads the same as one in $\mathrm{mol\,L^{-1}}$, where the thin space is kept inside the upright letters, and an energy flux is $3.5\,\mathrm{kg\,m^{2}\,s^{-3}}$. A number of five digits or more is split into groups of three by thin spaces, the integer part from the right and the decimal part from the marker: $12\,345$, $1\,234\,567.123\,456\,7$ and $98\,765.4321$. A number with an exponent gets a power of ten: light travels at $2.998 \times 10^{8}\,\mathrm{m\,s^{-1}}$, a mole holds $6.022\,14 \times 10^{23}$ particles, a millimetre is $1 \times 10^{-3}\,\mathrm{m}$, a small rate 3.5×10^{-4} and a charge 1.602×10^{-19}. The old font command `\rm` still works in a formula, where it switches to the upright family until the end of the group: $V_{\mathrm{NaOH}} = 24.57\,\mathrm{mL}$.

16.12 Program listings

pdfLaTeX's listings package reads a listing character by character, with a macro for each class of character, and decides keywords, comments and strings as it goes. RushTeX reads the listing's raw text once and builds the same boxes, one per character in fixed columns, for the keys templates use; any other key declines. The listings package sets program text line by line, each character of a line in a box of the same width, so that the columns of the source stay aligned:

```
<chapter id="opening">
  <title>The Opening</title>
    <para>Two  spaces, then one.</para>
</chapter>
```

A long line breaks where a token ends, and the rest goes on indented under it, as here:

```
result = combine(first_argument, second_argument,
    third_argument, fourth_argument, fifth_argument)
```

With `columns=fullflexible` the characters keep their own widths, a caption numbers the listing, and a language prints its keywords in a style of their own:

Listing 1: Sums of squares.

```
def sum_squares(n):
    total = 0
    for k in range(1, n + 1):
        total += k ** 2

    return total
```

The text after a listing starts a paragraph of its own only after a blank line.

16.13 Numbered lines, short code and other languages

Lines may be numbered in the margin, on the left or the right, each number in a style of its own; an empty line keeps its number:

```
int sum(const int *v, unsigned n) {
    int s = 0;

    for (unsigned i = 0; i < n; i++) s += v[i];
    return s;
}
```

```
for f in *.txt; do wc -l < f; done
```

A short piece of code sits in the running text, as `x = f(y, z)` does here, or with a language and a delimiter of its own: `public static void main()`. Keywords of C++, Java, the shell and MATLAB are printed in the keyword style as well:

```
x = linspace(0, 2*pi, 50);
plot(x, sin(x)); grid on
```

16.14 Comments and strings in listings

A language's comments and strings have styles of their own; a string shows its spaces unless `showstringspaces` is false, and a comment may run over several lines:

```
def greet(name):  # say hello
    return "Hello, " + name + '!'
```

```
/* A class with one
   field. */ class Note { String text = "to do"; } // end
```

In running text a string keeps its spaces too: `s = "a b"; //c`.

16.15 Named listing styles

A set of keys may be given a name and used by it, alone or with more keys:

```
x = y / 2; /* half */
```

```
return x; // done
```

The space around a listing is `aboveskip` and `belowskip`, the `\medskipamount` unless set to another length or skip; the keys that templates copy at their defaults (`showspaces=false`, `showtabs`, `extendedchars`, `keepspaces=false`, `upquote=false`, `captionpos`) change nothing here:

```
x = 1;
```

16.16 Listings read from a file

A listing may be read from a file of its own, here the source of the dummy figure, its long lines broken:

```
% A dummy picture for docs/master.tex: a frame with three
    nested boxes, the shape of
% the box tree \includegraphics builds. Made with: pdftex
    dummy.tex
\pdfpagewidth=240pt \pdfpageheight=120pt \pdfhorigin=0pt \
    pdfvorigin=0pt
\pdfcompresslevel=9 \pdfminorversion=5
\nopagenumbers \parindent=0pt \hsize=240pt \vsize=120pt
\def\frame#1#2#3{\vbox{\hrule height#3\hbox{\vrule width#3\
    vbox to#2{\vss\hbox to#1{\hss}\vss}\vrule width#3}\hrule
    height#3}}
\setbox0\hbox{\frame{232pt}{112pt}{2pt}}
\setbox1\hbox{\frame{180pt}{80pt}{1pt}}
\setbox2\hbox{\frame{120pt}{48pt}{0.6pt}}
\vbox to120pt{\offinterlineskip
\hbox to240pt{\hss\copy0\hss}
\kern-100pt
\hbox to240pt{\hss\copy1\hss}
\kern-66pt
\hbox to240pt{\hss\copy2\hss}
\kern-36pt
\hbox to240pt{\hss\tenbf dummy picture\hss}
\kern4pt
\hbox to240pt{\hss\sevenrm image inside two boxes\hss}
\vss}
\bye
```

The file is read at each compile, so editing it changes the listing.

16.17 Preprocessor lines and tabs in listings

A C line starting with # is a directive, its name in the keyword style; a tab moves the text to the next tab stop, every `tabsize` columns:

```
#include <stdio.h>
#define N 10
int main(void) {
    for (int i = 0; i < N; i++) /* count */
        printf("%d\n", i);
    return 0;
}
```

17 Writing the PDF

17.1 Shipping a page

When a page is complete, TEX ships it out: it walks the page's boxes from the top left, and for each glyph, rule and picture writes the PDF operators that place it, changing the current font whenever the next glyph needs another. pdfTeX writes the operators into a content stream for each page, compresses it, and at the end writes the fonts, the pictures and the cross-reference table that lets a reader find each object. RushTeX's shipping stage walks the same boxes, a word at a time, makes a font current once per word, and writes the same operators in the same order, so that the two files hold the same objects.

17.2 Fonts in the file

A PDF must carry the outline of every glyph it shows. pdfTeX reads each Type 1 font program from TEX Live, decrypts it, keeps only the glyphs the document uses, a *subset*, encrypts it again and embeds it. RushTeX carries the font programs in the program itself, already decrypted, so it reads no font file; it cuts the subset as pdfTeX cuts it, glyph for glyph, and gives it the same kind of six-letter tag.

17.3 Compression

Page contents and fonts are compressed with the deflate method [6], which codes repeated strings by reference and then codes the result with Huffman codes. pdfTeX uses the zlib library, which builds a Huffman code fitted to each stream. RushTeX chooses its encoder by how it is run. A one-shot compile, whose PDF is kept or sent, uses a strong encoder with fitted codes, and its files are as small as pdfLaTeX's. A compile inside a `--watch` session, whose PDF is replaced at the next save, uses the fixed code the method defines, which costs no work to build: measured on 2 November 2026, its files were about a seventh larger, and the compile did about a tenth less work. Compression does not change what is drawn, so the two files show the same pages.

18 Speed: each piece of work once

Exact output fixes what must be computed: TEX's expansion of the author's macros, the line breaker with its hyphenation, the page builder, the position of every glyph and the compression of every stream. Each of those runs near its least possible cost already, so a single compile on one processor core cannot be made arbitrarily faster. RushTeX gains its remaining speed in two other ways: it does not repeat work an earlier compile has done, and it gives independent work to several cores.

18.1 Compiling again after an edit

An author compiles a document many times, and each edit changes a few words. With `rushtex --watch` the program stays running and recompiles whenever the file is saved. pdfLaTeX starts from nothing at every run, loading its format and reading every package, and runs again while the auxiliary files change. RushTeX keeps a session.

- A save that changes nothing the document reads returns the last PDF at once.
- The pre-scan leaves a mark every 2048 bytes of the source, and after an edit it starts again from the last mark before the change, stopping as soon as its state equals the previous run's.
- The native job takes a checkpoint of its state as pages are shipped, every fourth page, as copy-on-write chunks of four kilobytes, so that a checkpoint costs a few small copies. After an edit, typesetting resumes from the last checkpoint before the first changed byte.
- When a later page leaves the job in the same state as the previous run did at that point, the rest of the document cannot have changed, and the compile stops there: the old pages are reused. Any doubt, such as a picture or a `.bbl` file that changed on disk, turns this early stop off.
- The compressed stream of each page and the subset of each font are kept, keyed by their content, so an unchanged page is neither walked nor compressed again.

Measured on 2 November 2026, on the 59-page version of this document, a save that adds or removes a word reached a new PDF in about 17 milliseconds, doing about 35 million instructions of work against about 380 million for the first compile of the session. A session's output is checked against a fresh compile: a fuzzer of random edits compares every recompile with a one-shot run byte for byte, and while the author is idle the session compiles the document afresh on another thread and compares the two.

A document on the engine path is resumed too. The engine copies its whole state at the end of the preamble and again where LATEX is about to read the auxiliary file, keyed by everything read before that point. A later compile whose preamble is unchanged starts from the copy and typesets the body alone, with four to seventeen times fewer instructions. The copies are kept on disk between compiles, written on a second thread as differences from one shared copy of the format, a few megabytes per document.

18.2 Several cores

TEX's state flows from each paragraph into the next, through counters, marks, floats and the depth of the last line, so typesetting is sequential by nature and RushTeX does not split it across threads. The work after it is independent: once the pages are set, the content stream of every page and the subset of every font are compressed, and the fonts encrypted, by a set of worker threads, each with its own compressor, and the results are written in the order a serial run writes them. The file is byte for byte the same; only the time to make it is shorter.

19 Exactness and the testing framework

Identical output is checked, not assumed. The testing framework, a separate Rust program called parity-testing, compiles each document of a corpus with RushTeX and with pdfLaTeX and compares the two files object by object: glyphs by font, character and position, rules and paths by coordinates, links, outline entries and colours by value. Then both files are rendered at 300 dots per inch and compared pixel for pixel, as Section 8 quoted.

When a page drifts, the framework shows pdfTeX's own record of the boxes and glue above the drifting line, so that the first node that differs can be traced back to the rule that produced it. New native features are also compared at 150 and 600 dots per inch, and box by box against the engine path, which is itself identical to pdfLaTeX on every document of the corpus.

This document is the native path's own test, and four gates guard every change to RushTeX:

1. the pre-scan must accept this document for the native path;
2. `parity-testing zero` must report it identical to pdfLaTeX's final run, every glyph, path, colour and pixel;
3. its pages must be box for box identical to the engine path's, compared from TEX's own record of each page's boxes, glue, kerns and penalties;
4. the three hundred templates that compiled natively before the change must give the same bytes after it.

The corpus of 85 documents from CTAN must stay at zero shift on the engine path at the same time. A feature that cannot pass these gates is not approximated: it keeps declining, and the engine sets it.

20 Measurements

Speed is judged on documents of ten pages or more, against one run of pdfLaTeX, by counting the instructions each program retires, a count that does not change with the load of the machine. The benchmark also repeats a document's body one, four and ten times and fits a straight line through the counts, which separates the fixed

cost of a run from the cost of each further page. The numbers below were measured on 25 October 2026, on an Apple M5 with TEX Live 2026, running pdfLaTeX with `-interaction=batchmode` and no latexmk.

20.1 The benchmark documents

Table T1 gives the instructions of one run of each program on the three benchmark documents of Table T1. pdfLaTeX needed three runs for the final PDF of each of them; RushTeX needed one.

Document	Pages	RushTeX	pdfLaTeX	Factor
Mathematics	20	141.9M	3240.2M	22.8
Tables	29	205.9M	3816.3M	18.5
Prose	61	279.6M	3280.7M	11.7

Table T1: Instructions of one run, measured on 25 October 2026.

Over pdfLaTeX's three runs the factors are 72.6, 58.5 and 36.9. Peak memory is 11.5 to 17.3 megabytes for RushTeX against 29.1 to 30.1 for pdfLaTeX. RushTeX's files were then 1.1 to 1.2 times larger, made with the fixed code alone; one-shot compiles now use the strong encoder of Section 17, and their files are as small as pdfLaTeX's.

20.2 Fixed cost and cost per page

The fit separates the two costs. The column specification of the next table removes the padding at both ends with an empty at-sign argument and repeats one column pattern with a star. Its rules are the booktabs package's: a heavy top and bottom rule, a light middle one, a partial rule under the last two headings trimmed at both ends, and a little extra space before the last row. The first heading spans both heading rows, with the multirow package, centred between them.

Repeats	Size	
	Pages	RushTeX
1	61	279.1M
4	242	875.8M
10	603	2063.5M

The prose document's fit gives the costs of Table T2.

The fixed cost is 28 to 45 times smaller across the three documents, mostly because RushTeX loads no format and reads no package or font file. The cost per page is 4.6 to 5.3 times smaller, so the factor falls as documents grow. An aim of forty times per page was set and then dropped on 27 October 2026: exact output means running TEX's own per-page algorithms, each already near its least cost, so one core tops out well below it. The speed beyond that comes from the two levers of Section 18: not repeating work after an edit, and giving the independent work to several cores.

Instructions of the prose document, from the fit		
Program	Per page	Fixed
RushTeX	3.3M	78.6M
pdfLaTeX	17.4M	2220.2M
Ratio per page		5.3

Table T2: Fixed and per-page cost, measured on 25 October 2026.

21 Current limits

The native path is young, and its limits are part of an honest description. Each limit below sends the whole document to the engine path, which sets it with the same output; none of them is approximated.

- Classes: beamer, the journal classes most papers use (revtex, elsarticle, llncs), CV classes such as moderncv, and every class or package file a document brings itself. Class options with behaviour of their own, such as `titlepage`, `leqno` and `fleqn` in article, also decline.
- graphicx takes PDF and PNG pictures; a JPEG picture runs on the engine.
- TikZ, babel, BibLaTeX and siunitx are native for the parts templates use most; a picture key, a language option or a citation style outside those parts declines.
- A page reference to a later page, a long table that runs over more than one page, a kernel internal redefined in the preamble, and a command the author defines with a text that could step a counter, used in a document with a contents list or a forward reference.
- The engine path needs an installed TEX Live.

The full list of the classes and packages the native path takes, with what each still declines, is `docs/native-packages.md` in the repository; it is read from the code, and a test fails when a native package is missing from it. For one document, `rushtex main.tex --native-gaps` names what keeps it off the native path. About three hundred of the 758 templates of the census compile natively today; the rest run on the engine path. Each limit is on the work list, in the order of the documents it would free.

A What this document tests

This appendix, numbered with letters after `appendix`, is the index of the test: it names what each section exercises, commands without their backslash, so that a change to RushTeX that breaks one shows which section to read.

Front matter `documentclass`, `usepackage` with options, `newcommand`, `renewcommand`, `def`, `newcounter`, `newcolumntype`, `newtheorem`, `title`, `author` with `thanks` and `and`, `date`, `maketitle`, `abstract`, `tableofcontents`, `listoffigures`, `listoftables`.

Sections 1 and 2 `section`, `subsection`, `emph`, `footnote`, forward `ref`, `enumerate` with a labelled item, nested `itemize`, `description`.

Section 3 the special characters, braces in Times and in typewriter type, the tie, accents and letters, UTF-8 input, quotes, dashes, `ldots`, the three skips, `em`, `ensuremath`, `rule`, `verb` and `verb*`, `MakeUppercase`, `uppercase`, `input`.

Section 4 a heading from the document's own macro, `label`, `ref`, `pageref`, `bibliography` from a `.bbl` file, the lists of figures and tables, preamble and body definitions, a command holding a centred line, `index`.

Section 5 every text font command and declaration, the two-letter forms, the ten sizes, `fontsize`, `textsuperscript`, `textsubscript`, kerns and ligatures, TS1 symbols, `copyright`.

Section 6 the space factor, `thinspace`, line breaks, `noindent`, `hspace`, `vspace`, `mbox`, hyphenation and `hyphenation`, `cite`, `newline`, `linebreak`, `nolinebreak`, `nopagebreak`, discretionary hyphens, `raggedbottom`, `enlargethispage`, setspace.

Section 7 `parbox` and `minipage` with and without a height, `fbox`, `colorbox`, `fcolorbox`, `makebox`, `framebox`, `raisebox`, `fill`, `hbox`, `raise`, `lower`, `rlap`, `llap`, `kern`, `vrule`, `vskip`, `sloppy`, `strut`, `hrulefill`, `dotfill`.

Section 8 nested lists, enumitem keys, short labels, `setlist`, description styles, `quote`, `quotation`, `verse`, `center`, `flushleft`, `flushright`, `centering`, `raggedright`, `raggedleft`, `newenvironment`.

Section 9 `subsubsection`, `paragraph`, `thispagestyle`, fancyhdr, `leftmark`, `markright`, `markboth`, a prefixed `thetable`, `refstepcounter`, `stepcounter`, `addtocounter`, the counter forms, `settowidth`, `stretch`, a footnote with an emphasis.

Section 10 `table` and `figure` at `h`, `t`, `b`, `htbp` and `H`, `figure*`, `caption` short and long, `clearpage`, `multicols` in two and three columns, `columnseprule`, `columnbreak`, columns in a box.

Section 11 `tabular` with rules, `hline`, `cline`, `tabularnewline`, `multicolumn`, `p`, `m` and `b` columns, tabularx, preamble declarations, `newcolumntype`, rows from a command, `verb` in a cell, colour in a cell, threeparttable, booktabs, longtable.

Section 12 `includegraphics` of a PDF and a PNG, TikZ pictures on the page and in the text, paths, nodes, arrows, loops, arcs, grids, opacity, `tikz`, qrcode, tcolorbox.

Section 13 inline and display math, fractions, roots, indices, Greek letters, symbols, operators with limits, named functions, the math alphabets, `left` and `right`, `equation`, `eqnarray` and `eqnarray*`, `notin`, `cong`, `underbrace`, `overbrace`, bm.

Section 14 `text`, `operatorname`, the dots, the `big` family, `equation*`, `notag`, `tag`, `eqref`, `align`, `gather`, `multline`, `aligned`, `gathered`, `split`, `cases`, the matrices, `array`, `boldsymbol`, `substack`, the extensible arrows, mathtools, display breaks.

Section 15 `newtheorem` with a reset level, a shared counter and a theorem's counter, `theoremstyle`, `newtheorem*`, `proof`.

Section 16 package options with no effect, siunitx loaded, geometry, xcolor, `url`, `path`, `urlstyle`, siunitx's `SI`, `si`, `num` and `S` columns, listings.

Sections 17 to 21 a table with an at-sign argument and a starred pattern, `multirow`, `cmidrule`, `addlinespace`, a table with a caption.

Section C `bibliography`, `bibliographystyle` and the `thebibliography` they bring.

B Feature probes

Features made native since the book was last reorganised are added here first, each as a subsection that explains it and uses it, and are folded into their chapter at the next reorganisation.

B.1 More of amsmath and mathtools

Pairs of columns set with no space between them, as `alignat` does,

$$x = 1 \qquad y = 2 \tag{B.1}$$
$$x + y = 3\, x - y = -1, \tag{B.2}$$

and pushed to the margins, as `flalign` does:

$$f(x) = x^2 \qquad\qquad g(x) = x^3.$$

Text may stand between the rows of an alignment:

$$u = v + w$$

and therefore, squaring both sides,

$$u^2 = v^2 + 2vw + w^2.$$

Equations grouped under one number,

$$\dot{x} = y, \tag{B.3a}$$
$$\dot{y} = -x, \tag{B.3b}$$

are (B.3), the first of them (B.3a), after (B.1). Double and triple integrals, continued fractions, boxed formulas, delimiters made in the preamble and overlapping limits:

$$\iint_D f\,dA + \iiint_V g\,dV, \quad \frac{1}{1+\dfrac{1}{2}}, \quad \boxed{\left|\frac{a}{b}\right| = |c| + |d|}, \quad \sum_{1 \le i \le n} x_i.$$

Under the hyperref package the group of equations gets its own anchor where it begins, for a reference to the whole group, and each equation in it an anchor named after the group's and its letter, so that a link to (2a) and a link to (3a) never lead to the same place.

A map $d\colon X \to [0, \infty)$ takes values in a half-open interval.

Proof. The last display ends the proof:

$$\overline{AB} = \overline{BA}. \qquad \square$$

B.2 Scaled, resized and rotated boxes

Wide tables and labels are often squeezed or turned with graphicx: `scalebox` multiplies a box by a factor, `resizebox` computes the factor from a size asked for, and `rotatebox` turns a box about its reference point. pdfLaTeX runs the macros of graphics.sty, which divide lengths by halving loops and compute sines with a series in TEX's own arithmetic; RushTeX does the same arithmetic in Rust on the box itself, so the sizes and the transformation matrices in the PDF are the same, without running a macro: here bigger, boom, fitted to two centimetres, up and tilted. The table below is resized to the text width.

Model	A	B	C	D	E	F
Baseline	0.81	0.82	0.83	0.84	0.85	0.86
Ours	0.91	0.92	0.93	0.94	0.95	0.96

A picture can be turned, trimmed and clipped as it is included, and found through `graphicspath`, which here names the folder of the manual's figures: 

and



B.3 Cells of several lines

The makecell package breaks a table cell into lines with `makecell`, and sets column heads in a smaller font with some room above and below with `thead`; each is a small tabular of its own inside the cell. A `multirow` entry may also be given a width, in which its text is set as a paragraph. RushTeX builds these nested tables with its own alignment code, as LaTeX does.

Model	Accuracy (%)	Time (s)
Ours (small)	85.4	15
Ours (large)	88.9 $\pm$0.3	41
A group with a long name	1 3	2 4

B.4 Spaces after commands and float barriers

A command that stands for a word, such as `\rtname` here, swallows the space after it; xspace's `xspace` puts the space back unless punctuation follows: RushTeX is fast, and so is RushTeX. pdfLaTeX looks at the next token with `futurelet`; RushTeX looks at it directly. The nth package writes ordinals, here with its `super` option: the 1st, 2nd, 3rd, 11th and 22nd edition, the suffix chosen from the last two digits. The placeins package's `FloatBarrier` keeps floats from drifting past it: the floats still held are put out first, as here before the next subsection. (indentfirst, which indents the first paragraph after every heading, would change the whole manual and is tested on its own.)

B.5 Commands with a default argument

A command's first argument may be optional, with a default: $(x_1, \ldots, x_n)$ has n parts and $(y_1, \ldots, y_m)$ has m; we say hello, world and hello, reader.
Remark. *An environment may have one too.*
Aside. *Given here.*

B.6 The title read again

A title page made by hand reads the title through `\@title`, `\@author` and `\@date` after `\makeatletter`. `\maketitle` empties them, so here they give nothing: [].

B.7 Switches of the document's own

A switch made with `\newif` chooses text: this book is final; here it is a draft only inside the group.

B.8 Internal macros of the document's own

Templates often name their helpers with an `@` after `\makeatletter`, as the kernel names its own. Such a macro is the document's when neither the kernel, nor the class, nor a loaded package has a macro of that name: defining it then changes nothing they read. RushTeX keeps a list of every `@` name pdfLaTeX's kernel, the standard classes and the packages it loads define or build, made from their sources and from LuaTEX's table of names after loading each one, and takes a definition of any other name as the document's own: **left /** right. A definition of a name on the list, such as `\@seccntformat`, still sends the document to the engine.

B.9 Headings of the document's own

B.9.1 A heading with *own* macros

A heading may hold a macro of the document's own, as this one does: the contents line `\addcontentsline` writes is the macro's expansion, the same text the contents get when they expand the macro themselves, so a macro defined once in the preamble from commands a contents line may hold takes its place in the table of contents above. `\setcounter{secnumdepth}{0}` in the preamble leaves the headings unnumbered, their contents lines without numbers and their running heads without them; a heading past the depth keeps its own link target under hyperref. This manual keeps its numbers, so the setting is only described here.

Run-in headings in a row.

Then another. A run-in heading followed at once by another gets a paragraph of its own, as `\@startsection`'s `\if@noskipsec \leavevmode \fi` gives it. The tables of this manual are numbered within their section, as `\counterwithin{table}{section}` asks (table T1 below); a macro of the document's whose text starts with `\shortintertext` works in an alignment, and an interjection may stand in the middle of a row, which it ends:

$$a = b + c$$

so that

$$a - b = c$$

and therefore

$$c = a - b$$

one	two

Table T1: A table numbered within its section

B.10 Anchors, links and running heads

A heading past the depth set in the body

`\setcounter{secnumdepth}{2}` set in the body leaves the next headings at the third level unnumbered, as this one is, here and in the contents; setting it back to 3 numbers the following ones again, and the contents read each heading with the depth as it stood.

B.10.1 A heading numbered again

Under hyperref an equation's `\tag` is a link target of its own, `AMS.`n from hyperref's link counter; in an `align` or `gather` the measuring pass takes the counter once for each tag before the rows take it again, so the targets go up in twos there, and a `\label` on the row points at the tag. A URL given to `\href` may hold `#`, `~` and `\%`: `\#` and `\%` become their characters, and the part after the `#` is the target's fragment. A `\markboth` of the document's own macros, `\ ` , and `\texttt` keeps the macros' text as `\protected@xdef` expands it, under the `myheadings` style, which lets `\sectionmark` gobble its argument whatever `secnumdepth` is; book's `headings` drops the number from a section's mark at depth 0. With hyperref, `\counterwithin{figure}{section}` makes `\theHfigure` the section's number and the figure's, so the captions' targets are `figure.1.1`, `figure.1.2`.

B.11 Tags read ahead, tags in multline, and old font commands

A reference may come before a tag made of a formula or commands, as $**$ and († M) do here: the value is the tag's own tokens, read from the source the way pdfLaTeX's last run reads them back from the `.aux` file, and checked against the tokens when the label is met. A `\tag` may stand in any row of a `multline`, which sets it beside the last row; under hyperref it is a link target too, the counter going up twice as in `align`, and with microtype the tag's parentheses protrude as microtype's patch of `\tagform@` asks.

$$a + b + c + d \\ = e + f \quad (\dagger\,\mathrm{M})$$

$$\mathcal{L}(x) = \begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix} \qquad (**)$$

The class's old `\cal` sets the calligraphic family for the rest of its group in math (and does nothing in text), and a macro of the document's may hand an `array` its preamble as a parameter, as `\rtmat` does above.

B.12 Bare alignment commands, one-token definitions, math in bookmarks

A definition's text may be a single token without braces, `\newcommand\rteps \epsilon`, as LaTeX reads it as an argument: here $\epsilon < 1$. Homework headers often write `\flushleft` or `\flushright` as a command at the top of a minipage instead of the environment; the trivlist it starts is never ended, and the minipage's group undoes it.

Course notes, chapter 2: exercises 8.8, 8.9, 8.10 & 10.3 — A. Student

Reading: sections 9.1 and 10.1

With hyperref, a heading may hold math made of letters and the symbol commands (`\subsection{Expansion in $\beta$}`): the bookmark drops the math shifts, scripts, braces and symbols, as `\pdfstringdef` does, and keeps every space.

B.13 Conditions with ifthen

The ifthen package writes the same choices in words: `\newboolean` makes a switch, `\setboolean` sets it to `true` or `false`, and `\ifthenelse` takes a test and two branches, of which it keeps one. A test compares numbers and counters, asks whether one is odd, compares two texts after expanding them, reads a boolean, and joins such tests with `\AND`, `\OR` and `\NOT`. This is section B, so the book is not yet long; its number is even; the language macro reads en, and the answers are shown. After `\setboolean{rtanswers}{False}` they are hidden. RushTeX evaluates the test as the package's chain of `\ifnum` tests does, left to right, and pushes the branch it keeps back to be read, so the branch is typeset as LaTeX typesets it.

B.14 Page styles of the document's own

fancyhdr's `\fancypagestyle{plain}{...}` gives the `plain` page style, the one `\maketitle`, a chapter's first page and `\thispagestyle{plain}` ask for, heads and feet of the writer's own; this book's preamble gives it a small title at the right of the head, the page number centred in the foot and no rule under the head. pdfLaTeX stores the definitions in `\ps@plain` and runs them, with `\ps@fancy`'s settings, in the output routine of each such page. RushTeX runs the slot commands once, in the preamble, keeps the slots they set beside the `fancy` ones and the rule widths to set in the page's output group, and builds the head and foot of a `plain` page from them: the same boxes, with no macro run per page. This page is one.

C References

The algorithms of the native path are those of the standard references. The paragraph builder is the method of Knuth and Plass [1]; the page builder, the output routine and the mathematics follow the reference manual of TeX [2] and its program [3]; the hyphenation patterns follow Liang's thesis [4]; LaTeX's commands follow its manual [5] and its sources; compression follows the deflate specification [6]. The chain of stages

in Figure 1 is a common design for compilers in general, as Aho and colleagues describe [7].

Each entry's label is `\@biblabel` of its number, `[1]` in the standard classes; many documents redefine it after `\makeatletter`, as `\renewcommand\@biblabel[1]{#1.}`. RushTeX builds the label itself while the macro is the kernel's, and otherwise places the document's macro in the label's box and in the measurement of the widest label, where LaTeX expands it. This document redefines it to the kernel's own text, so the list below looks as before and exercises the path.

References

[1] Donald E. Knuth and Michael F. Plass. *Breaking paragraphs into lines*. Software: Practice and Experience, 11(11), 1981.

[2] Donald E. Knuth. *The TeXbook*. Addison-Wesley, 1986.

[3] Donald E. Knuth. *TeX: The Program*. Addison-Wesley, 1986.

[4] Franklin M. Liang. *Word Hy-phen-a-tion by Com-put-er*. PhD thesis, Stanford University, 1983.

[5] Leslie Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, second edition, 1994.

[6] L. Peter Deutsch. *DEFLATE Compressed Data Format Specification version 1.3*. RFC 1951, 1996.

[7] Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, second edition, 2006.